

운영체제 8

남궁명수

[1-1] 분산 시스템

핵심 질문: 구성 요소가 실패하더라도 동작하는 시스템을 어떻게 만들까

- 가끔씩 고장나는 부품들로 어떻게 항상 동작하는 시스템을 만들까?
- RAID 에서 다루었던 주제들이 기억날 것이다.
- 하지만 여기서 다루는 문제는 좀 더 복잡하다. 해법마저도 복잡하다.

분산 시스템은 단순히 여러 대의 컴퓨터를 사용하는 것이 아니라, 여러 기계가 하나의 시스템처럼 동작하도록 만드는 구조이다.

예를 들어, 웹 브라우저로 어떤 사이트에 접속한다고 생각해보자.

우리는 그냥 “하나의 웹사이트”를 사용하는 것처럼 느끼지만, 실제로는 지구 어딘가에 있는 서버와 통신하고 있는 것이다.

더 중요한 건, 그 서버가 한 대가 아니라는 점이다.

예를 들어,

- 검색 서비스를 제공하는 시스템
- 소셜 네트워크 서비스

이런 것들은 실제로:

수천 대의 서버들이 협력해서 하나의 서비스처럼 동작

즉, 사용자는 “하나의 서비스”를 쓰지만 실제 내부는 “엄청나게 많은 기계들의 협력 구조”다.

여기서 분산 시스템이 흥미로운 이유가 나온다.

우리는 완벽한 컴퓨터를 만들 수 없다.

- 디스크는 고장난다
- 네트워크는 끊긴다
- 서버는 다운된다.
- 소프트웨어도 버그가 있다.

이건 피할 수 없는 사실이다.

그런데도 우리가 사용하는 서비스는 이렇게 느껴지길 원한다.

“절대 멈추지 않는 시스템”

이게 바로 분산 시스템의 핵심 목표다.

그래서 분산 시스템의 가장 중요한 문제는 “어떻게 실패를 감추고, 시스템을 계속 동작하게 만들 것인가”이다.

예를 들어보자. 서버 1000 대가 있는 시스템에서

- 서버 1 대가 고장남 -> 문제 없음
- 서버 10 대가 고장남 -> 여전히 서비스 가능
- 일부 네트워크 장애 발생 -> 그래도 동작

즉, 개별 구성 요소는 계속 고장나지만 전체 시스템은 정상처럼 보이게 만든다.
이게 분산 시스템의 본질이다.

그 다음 중요한 문제가 성능이다.
여러 대의 기계를 쓴다고 해서 무조건 빠르게 아니다.

오히려 다음과 같은 문제가 생긴다.

- 네트워크 통신 비용이 큼
- 메시지를 주고받는데 시간이 걸림(지연 시간)
- 데이터가 여러 곳에 퍼져 있음

그래서 설계자는 항상 고민한다

“어떻게 통신을 줄일 것인가?”
“어떻게 빠르게 응답할 것인가?”

예를 들어:

- 메시지를 10 번 보내야 하는 구조 -> 느림
- 메시지를 1 번으로 줄이면 -> 빠름

즉, 네트워크 비용이 성능의 핵심 요소다.

그리고 또 하나 빠질 수 없는게 보안이다.

분산 시스템은 기본적으로 네트워크 위에서 동작하기 때문에 다음과 같은 문제가 생긴다.

- 내가 접속한 서버가 진짜 서버인가?
- 데이터가 중간에 변조되지는 않았는가?
- 누군가 통신 내용을 훔쳐보지 않는가?

즉, 신뢰할 수 없는 환경에서 통신해야 한다.

그래서 인증, 암호화, 무결성 검증 같은 기술이 필수로 들어간다.

이제 마지막으로 이 장의 핵심 질문이 나온다.

“그럼 여러 기계는 서로 어떻게 통신하는가?”

분산 시스템에서 기본 단위는 메시지(message)이다.

즉,

- 한 기계가 다른 기계에서 요청을 보낸다
- 응답을 받는다

이 단순한 구조 위에

- RPC
- HTTP
- 분산 프로토콜

같은 복잡한 시스템이 쌓인다.

그리고 여기서 다시 핵심으로 돌아온다.

분산 시스템에서 가장 중요한 관점은 “통신은 실패할 수 있다”

- 메시지가 도착하지 않을 수도 있고
- 중간에 유실될 수도 있고
- 늦게 도착할 수도 있다.

그래서 앞으로의 내용은 결국 이걸 다룬다.

“실패하는 환경에서 어떻게 통신을 설계할 것인가?”

[1] 통신의 기본

이 부분에서 말하려는 핵심은 “네트워크는 기본적으로 신뢰할 수 없는 환경이다”라는 전제를 받아들이라는 것이다. 우리가 보통 프로그램을 작성할 때는 함수 호출을 하면 반드시 실행되고 결과가 돌아온다고 생각하지만, 분산 시스템에서의 통신은 그런 식으로 동작하지 않는다.

한 기계가 다른 기계로 메시지를 보내는 순간부터, 그 메시지가 어떻게 될지는 보장할 수 없다.

메시지는 네트워크를 따라 여러 장비를 거쳐 이동한다. 이 과정에서 다양한 문제가 발생할 수 있다. 전송 중에 전기적 노이즈나 하드웨어 문제로 비트가 바뀌어 데이터가 손상될 수 있고, 중간에 있는 라우터나 스위치 같은 장비가 고장 나거나 오작동할 수도 있다. 심지어 물리적으로 케이블이 끊기는 상황도 완전히 배제할 수 없다. 이런 경우 메시지는 아예 목적지에 도달하지 못한다.

그보다 더 흔하고 중요한 원인은 네트워크 장비나 호스트의 자원 한계다.

메시지는 네트워크를 통해 이동할 때 단순히 “흐르는” 것이 아니라, 각 라우터나 수신 측 시스템의 메모리(버퍼)에 잠시 저장되었다가 처리된다.

그런데 한 번에 너무 많은 패킷이 물리면 이 버퍼가 가득 차게 된다.

라우터 입장에서는 모든 패킷을 저장할 수 없기 때문에 일부를 버릴 수밖에 없다. 이걸 패킷 드롭이라고 한다. 이 현상은 네트워크 장비뿐 아니라 최종 목적지인 서버에서도 똑같이 발생한다. 서버가 처리할 수 있는 속도보다 더 많은 요청들이 들어오면, 일부 요청은 큐에 쌓이지 못하고 그냥 사라진다.

중요한 점은, 이런 상황이 “예외적인 사건”이 아니라는 것이다. 네트워크에서는 패킷 손실, 지연, 손상은 정상적인 현상이다. 즉, 메시지가 한 번 보내졌다고 해서 반드시 도착한다고 가정하는 설계는 잘못된 것이다. 이 전제를 받아들이지 않으면 분산 시스템은 반드시 깨진다.

그래서 자연스럽게 다음 질문이 나온다.

“그렇다면 이런 환경에서 어떻게 통신을 설계해야 하는가?” 단순히 메시지를 보내는 것만으로는 충분하지 않다. 메시지가 사라질 수 있다는 가정 하에, 재전송, 타임아웃, 확인응답 같은 메커니즘을 추가로 설계해야 한다. 결국 분산 시스템에서의 통신은 “신뢰할 수 없는 기반 위에서 신뢰를 만들어내는 과정”이라고 볼 수 있다.

[2] 신뢰할 수 없는 통신 계층

여기서 말하는 핵심은 “UDP 같은 신뢰할 수 없는 통신 계층이 실제로 어떤 의미냐”를 코드까지 포함해서 이해하는 것이다.

단순히 “패킷이 유실될 수 있다”라는 수준이 아니라, 프로그래머 입장에서 어떤 보장이 업속, 무엇을 직접 처리해야 하는지가 중요하다.

우선 UDP는 기본적으로 “그냥 보내는” 통신이다.

클라이언트가 `sendto()`로 메시지를 보내면, 네트워크는 그 메시지를 전달하려고 시도는 하지만 도착을 보장하지 않는다. 중간에 패킷이 사라지든, 순서가 뒤바뀌든, 중복되든, 송신자는 알 수 없다.

코드를 보면 그 특징이 그대로 드러난다.

클라이언트 쪽 흐름을 보면

- `UDP_Open(20000)` -> 소켓을 하나 만든다(포트 20000)
- `UDP_FillSockAddr()` -> 서버 주소 설정
- “hello world” 메시지를 만들어서
- `UDP_Write()` -> 서버로 전송

그리고 나서:

- `UDP_Read()`로 응답을 기다린다.

여기서 중요한 점은, 이 코드에서 “정말 도착했는지 확인하는 로직”이 없다. 그냥 보내고, 오면 받고, 안오면 끝이다.

서버 쪽도 마찬가지다.

- `UDP_Open(10000)` -> 서버 포트 열기
- 무한 루프 돌면서
- `UDP_Read()`로 메시지 기다림
- 메시지 오면 “reply”만들어서 `UDP_Write()`로 응답

이 구조만 보면 아주 단순한 요청-응답 시스템처럼 보인다.
하지만 실제로는 다음과 같은 상황이 모두 가능하다.

1. 클라이언트가 메시지를 보냈는데 서버가 못 받는 경우
2. 서버가 받았는데 응답이 중간에서 사라지는 경우
3. 클라이언트가 동일한 요청을 여러 번 보내는 경우
4. 응답이 두 번 도착하는 경우
5. 응답이 늦게 와서 이미 타임아웃 처리된 뒤 도착하는 경우

이 모든 상황을 UDP 는 전혀 처리해주지 않는다.
그럼에도 불구하고 UDP 가 존재하는 이유는 단순하다.
“아무것도 보장하지 않는 대신 매우 가볍고 빠르다”

즉

- 연결 설정 없음(TCP 의 3-way handshake 없음)
- 상태 관리 없음
- 재전송 없음
- 흐름 제어 없음

그래서 지연(latency)이 중요하거나, 약간의 손실을 허용할 수 있는 경우에 적합하다.
예를 들어,

- 실시간 스트리밍
- 온라인 게임
- DNS 요청

이런 곳에서는 “완벽한 전달”보다 “빠른 전달”이 더 중요하다.

하지만 대부분의 애플리케이션은 그렇지 않다.
우리는 보통 다음을 원한다.

- 메시지가 반드시 도착해야 한다.
- 중복 없이 한 번만 처리되어야 한다.
- 순서가 유지되어야 한다.

UDP 는 이것 전혀 제공하지 않는다. 그래서 그 위에 신뢰성을 직접 구현해야 한다.
예를 들어, 간단한 보안을 생각해보면 클라이언트가 메시지를 보낼 때

- 메시지에 ID 를 붙인다.(예: msg_id=1)
- 메시지 전송 후 일정 시간 기다린다
- 응답이 없으면 다시 보낸다(재전송)

서버는:

- 이미 처리한 msg_id 인지 확인
- 중복이면 다시 응답만 보내고 처리 안함

이렇게 하면 최소한 “유실”과 “중복” 문제는 어느 정도 해결된다.

또 하나 중요한 점은 UDP 도 완전히 무방비는 아니라는 것이다.

코드 설명에서도 나오듯이 UDP 는 체크섬을 포함한다. 즉, 패킷이 전송 중에 손상되면 어느 정도는 검출할 수 있다.

하지만 여기서도 한계가 있다. 손상은 검출하지만 복구는 안해준다.

즉, 깨진 패킷은 그냥 버려진다.

결국 정리하면 이 계층의 본질은 이렇다.

- UDP 는 “최소 기능만 제공하는 통신 계층”이다.
- 데이터 전달은 시도하지만, 성공 여부는 책임지지 않는다
- 일부 오류(손상)는 감지하지만, 해결은 하지 않는다
- 따라서 신뢰성은 반드시 상위 계층에서 구현해야 한다

이걸 이해하는게 중요한 이유는, 이후에 나오는 TCP 나 분산 시스템의 다양한 프로토콜이 왜 그렇게 복잡한지를 이해하게 해주기 때문이다.

결국 그 복잡함은 전부 “신뢰할 수 없는 네트워크 위에서 신뢰를 만들어내기 위한 비용”이다.

[3] 신뢰할 수 있는 통신 계층

여기서는 “신뢰할 수 없는 UDP 같은 환경 위에서 어떻게 신뢰성을 만들어내는가”를 단계적으로 보여준다. 핵심은 간단하다. 네트워크는 믿을 수 없기 때문에, 우리가 직접 믿을 수 있게 만들어야 한다는 것이다.

가장 먼저 해결해야 하는 문제는 이것이다. “보낸 메시지가 정말 도착했는지 어떻게 아는가”이다.

이를 위해 등장하는 개념이 ack(acknowledgement)이다.

동작은 단순하다.

- 발신자가 메시지 M 을 보낸다
- 수신자는 메시지를 받으면 “받았다”라는 짧은 응답(ack)을 보낸다
- 발신자는 ack 를 받으면 전달 성공이라고 판단한다

예를 들어보면,

- 클라이언트 -> 서버: “A” 전송
- 서버 -> 클라이언트: “ACK(A)” 응답

이렇게 하면 “적어도 서버까지는 도착했다”는 사실을 확인할 수 있다.
그런데 여기서 바로 문제가 생긴다. ack 가 항상 도착한다는 보장이 없다.

그래서 등장하는 것이 타임아웃(timeout) + 재전송(retry)이다.
동작은 이렇게 바뀐다.

- 발신자가 메시지를 보내면서 타이머를 건다.
- 일정 시간 안에 ack 가 오지 않으면 -> “아, 이거 실패했나 보다”라고 판단하고 같은 메시지를 다시 보낸다.

예를 들어,

1. 클라이언트 -> 서버: “A”
2. 서버는 “ACK(A)”를 보냈는데 중간에서 유실됨
3. 클라이언트는 ack 를 못받음 -> timeout 발생
4. 클라이언트 -> 서버: “A” 다시 전송

여기까지 보면 꽤 그럴듯하게 동작하는 것처럼 보인다. 하지만 이 구조는 아직 문제가 있다.

문제의 핵심은 이거다.

수신자는 같은 메시지를 두 번 받을 수 있다.

위 상황을 그대로 이어보면:

- 서버는 처음 “A”를 이미 처리했다.
- 그런데 클라이언트가 다시 “A”를 보내서 -> 서버는 “A”를 또 받게 된다.

이건 심각하다.

예를 들어,

- “돈 1000 원 송금” 메시지가 두 번 처리되면?
- 파일에 같은 데이터가 두 번 추가되면?

그래서 신뢰성 있는 통신의 목표는 단순히 “도착”이 아니라 “정확히 한 번만 처리”이다.

이걸 해결하려면 수신자가 “이 메시지 처음 보는 건지, 전에 본 건지”를 구분할 수 있어야 한다. 그래서 등장하는 게 메시지 식별자(ID)이다.

가장 단순한 방법은

- 발신자가 모든 메시지에 고유 ID 를 붙인다.
- 수신자는 지금까지 받은 ID 들을 전부 기억한다
- 이미 받은 ID 면 -> 중복이므로 처리 안하고 ack 만 보낸다.

이 방식은 정확하지만 현실적으로 문제가 있다.

ID 를 계속 저장해야 해서 메모리가 무한히 필요하다.

그래서 더 현실적인 방법이 순서 카운트(sequence counter)다.
이건 아주 중요한 아이디어다.

동작을 보면:

- 발신자와 수신자가 처음에 “시작 번호”를 합의한다(예:1)
- 메시지를 보낼 때마다 번호를 붙인다.

예를 들어,

- 첫 메시지: (ID = 1, 데이터 = A)
- 두 번째 메시지: (ID = 2, 데이터 = B)

수신자는 “다음에 와야 할 번호”를 하나만 기억한다.

예를 들어, 수신자가 현재 기대하는 값이 1 이라면

- ID = 1 이 오면 -> 정상, 처리하고 다음 기대값을 2 로 변경
- ID = 2 가 오면 -> 정상
- ID = 1 이 다시 오면 -> 이미 처리된 것 -> 버리고 ack 만 보냄

이 구조가 중요한 이유는

- 저장해야 할 정보가 “하나의 숫자”뿐이다.
- 중복 메시지를 효과적으로 걸러낼 수 있다.

아까 ack 손실 상황을 이걸로 다시 보면:

1. 클라이언트 -> 서버: (ID = 1, A)
2. 서버: 정상 처리 후 ACK(1) 전송
3. ACK 가 유실됨
4. 클라이언트 timeout -> (ID=1, A) 재전송
5. 서버는 ID = 1 이미 처리했음을 앎 ->
 - 다시 처리하지 않음
 - ACK 만 다시 보냄

결과적으로,

- 클라이언트는 결국 ack 를 받는다.
- 서버는 메시지를 한 번만 처리한다.

이게 바로 신뢰성의 핵심 구조다.

정리하면 이 계층은 다음 세가지 요소로 구성된다.

1. ack -> “도착했는지 확인”
2. timeout + 재전송 -> “유실 복구”
3. sequence number -> “중복 제거”

이 세가지가 합쳐지면, 신뢰할 수 없는 네트워크 위에서도
 “적어도 한 번 전달 + 중복 제거 -> 사실상 한 번만 처리”를 구현할 수 있다.
 마지막으로 언급된 TCP 는 이 아이디어를 훨씬 더 확장한 것이다.

TCP 는 단순히

- ack, 재전송, 순서 번호

이 정도에서 끝나지 않고,

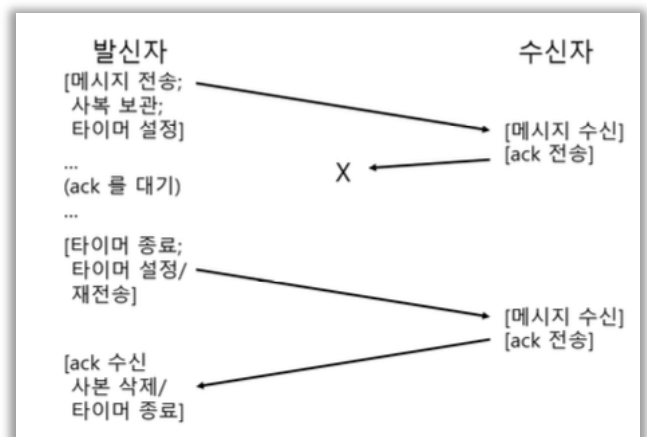
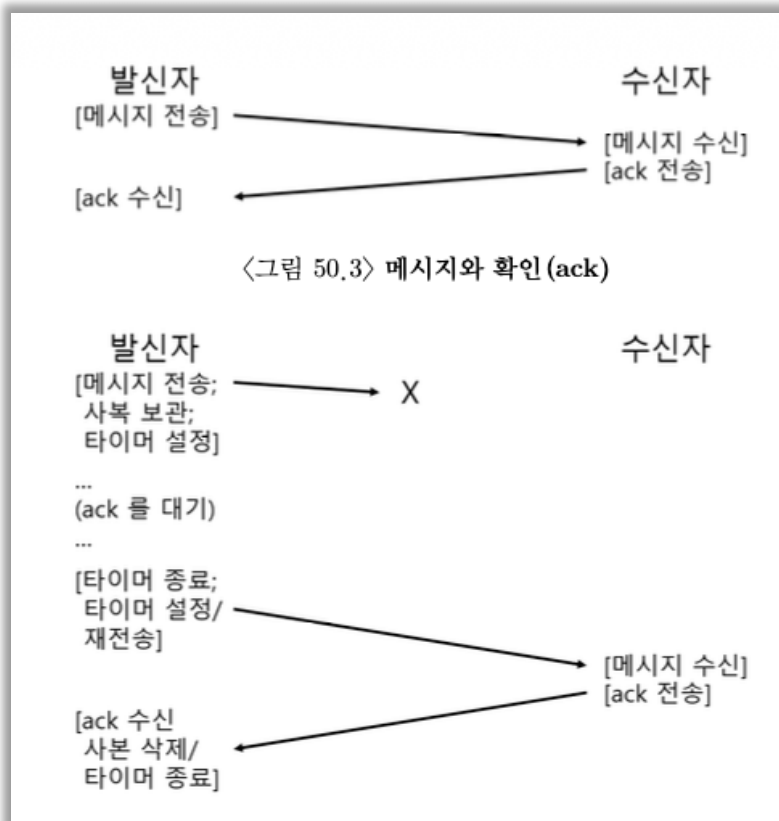
- 흐름 제어(receiver 가 감당 가능한 속도로 보내기)
- 혼잡 제어(네트워크가 터지지 않도록 속도 조절)
- 순서 보장(out-of-order 정렬)
- 버퍼링 및 파이프라이닝

같은 수많은 기능을 포함한다.

그래서 TCP 는 “신뢰할 수 있는 바이트 스트림”을 제공한다.

결국 중요한 포인트는 이것이다.

- 네트워크는 기본적으로 신뢰할 수 없다
- 신뢰성은 자동으로 생기는데 아니라
- ack, timeout, sequence 같은 메커니즘을 조합해서 만들어내는 것이다.



[4] 통신 추상화

여기서 말하려는 핵심은 “분산 시스템에서 통신을 어떻게 추상화할 것인가”이고, 그 중 하나의 시도로 나온 것이 DSM(Distributed Shared Memory)이다.

아이디어 자체는 굉장히 직관적이다. 원래 하나의 컴퓨터에서 여러 스레드가 하나의 메모리 공간을 공유하면서 동작한다.

DSM은 이 개념을 여러 대의 컴퓨터로 확장해서, 서로 다른 기계에 있는 프로세스들이 마치 하나의 거대한 메모리를 공유하는 것처럼 보이게 만들자는 접근이다.

즉, 프로그래머는 네트워크나 메시지를 직접 다루지 않고 그냥 메모리 읽고 쓰듯이 코드를 작성하면 되고, 운영체제가 그 뒤에서 알아서 데이터를 가져오거나 보내는 역할을 한다.

이게 가능한 이유는 가상 메모리 시스템 때문이다.

어떤 데이터에 접근했을 때 그 데이터가 현재 내 머신에 있으면 그냥 메모리에서 바로 읽으면 된다. 하지만 그 데이터가 다른 머신에 있다면 페이지 폴트가 발생하고, 운영체제가 네트워크를 통해 해당 페이지를 요청해서 가져온 뒤 메모리에 올리고 다시 실행을 이어간다. 겉보기에는 단순한 메모리 접근인데, 실제로는 네트워크 통신이 숨어 있는 구조다.

이론적으로 분산 시스템을 완전히 숨기고, 기존의 멀티스레드 프로그램처럼 쉽게 작성할 수 있게 해주는 매력적인 모델이다.

하지만 현실에서는 거의 사용되지 않는다. 가장 큰 이유는 실패 처리 때문이다.

분산 시스템에서는 언제든지 어떤 기계가 고장날 수 있는데, DSM에서는 데이터가 여러 머신의 메모리에 흩어져 있기 때문에 특정 머신이 죽으면 그 머신에 있던 메모리 내용이 통째로 사라진다.

문제는 이게 단순히 일부 데이터 손실이 아니라, 전체 데이터 구조를 망가뜨릴 수 있다는 점이다. 예를 들어, 연결 리스트나 트리 같은 구조에서 어떤 노드가 사라지면, 그 노드를 가리키던 포인터는 더 이상 유효하지 않게 된다.

결국 프로그램은 존재하지 않는 메모리를 참조하게 되고, 이는 복구하기 매우 어려운 상태로 이어진다. 일반적인 프로그램은 “메모리가 갑자기 사라진다”는 상황을 전혀 가정하지 않기 때문에 이런 문제를 자연스럽게 처리할 방법이 없다.

성능 문제도 매우 크다. 프로그래머는 보통 메모리 접근이 빠르다고 가정하고 코드를 작성한다. 그런데 DSM에서는 어떤 접근은 로컬 메모리라서 빠르지만, 어떤 접근은 다른 머신에 있는 데이터를 가져와야하므로 네트워크 통신이 발생하고 훨씬 느려진다.

문제는 이 차이가 코드 상에서는 전혀 드러나지 않는다는 점이다.

같은 코드인데 어떤 경우에는 빠르고, 어떤 경우에는 극단적으로 느려질 수 있기 때문에 성능을 예측하기가 매우 어렵다.

특히, 반복문이나 자주 호출되는 코드에서 원격 페이지 접근이 발생하면 성능이 급격히 떨어진다. 결국 프로그래머는 “이 데이터는 어디에 두어야 하는지”, “이 접근이 네트워크를 타는지”를 신경 쓰게 되는데, 이렇게 되면 DSM이 제공하려던 추상화의 의미가 사라진다.

결국 DSM은 “분산 시스템을 숨기자”는 방향이었지만, 실제로는 실패와 성능이라는 중요한 요소를 제대로 다루지 못했다.

그래서 현재의 분산 시스템에서는 이런 식으로 완전히 숨기는 접근을 거의 사용하지 않는다. 대신 통신과 데이터 이동을 명시적으로 드러내고, 메시지 기반이나 RPC 같은 방식으로 구조를 설계한다.

이렇게 하면 프로그래머가 비용과 실패 가능성을 인지하고 설계할 수 있기 때문에, 전체 시스템을 더 안정적이고 예측 가능하게 만들 수 있다.

[5] Remote Procedure Call(RPC)

RPC를 이해할 때 핵심은 “원격에 있는 코드를 마치 내 프로그램 안에 있는 함수처럼 호출한다”는 목표를 끝까지 유지하면서, 그 뒤에서 실제로 어떤 일이 벌어지는지를 따라가는 것이다.

단순히 개념만 보면 쉬워보이는데, 내부 동작을 보면 왜 이게 중요한 기술인지가 보인다.

우선 상황을 하나 잡고 시작해보자.

클라이언트 프로그램이 있고, 서버에는 어떤 함수가 있다고 하자.

```
int add(int a, int b);
```

이 함수는 서버에 존재한다. 그런데 클라이언트에서 이것을 이렇게 쓰고 싶다.

```
result = add(2, 3);
```

겉으로 보면 그냥 로컬 함수 호출이다. 하지만 실제로는 네트워크를 건너가야 한다. RPC의 역할은 이 “차이”를 숨기는 것이다.

이제 실제 내부 흐름을 순서대로 보면 이렇게 된다.

클라이언트 코드가 `add(2, 3)`을 호출하면, 실제로는 서버에 있는 함수가 바로 실행되는게 아니라 클라이언트 스텝(client stub)이 대신 호출된다.

이 스텝이 하는 일이 핵심이다.

1. 메시지 버퍼를 만든다.

- 예를 들어, 바이트 배열 같은 공간을 만든다

2. 함수 호출 정보를 그 안에 넣는다

- 어떤 함수인지(예: `add`)

- 인자 값들(`2, 3`)

이 과정을 마샬링(marshaling) 또는 직렬화(serialization)라고 한다.

예를 들어, 내부적으로는 이런 식으로 바꿀 수 있다.

```
[ function_id = 1 | arg1 = 2 | arg2 = 3 ]
```

3. 이 메시지를 네트워크로 서버에 보낸다.

4. 응답을 기다린다(여기서 클라이언트는 멈춰 있음)

이제 서버 쪽으로 넘어간다.

서버는 메시지를 받으면 서버 스텝스(server stub)이 먼저 처리한다.

서버 스텝스는 다음 일을 한다.

1. 메시지를 해석한다(역직렬화, unmarshaling)

```
function_id = 1 → add 함수  
arg1 = 2  
arg2 = 3
```

2. 실제 서버 함수 호출

```
result = add(2, 3); //실제 실행
```

3. 결과를 다시 메시지로 만든다

```
[ result = 5 ]
```

4. 클라이언트에게 다시 보낸다.

다시 클라이언트로 돌아온다.

클라이언트 스텝스가 응답을 받으면:

1. 메시지를 풀어서 값 추출

2. 호출한 코드에 결과 반환

결국 클라이언트 입장에서는

```
result = add(2, 3); // 그냥 함수 호출처럼 보임
```

이렇게 끝난다.

여기까지 보면 단순한데, 실제로는 몇 가지 중요한 문제가 숨어 있다.

첫 번째, 데이터를 어떻게 보내느냐다.

단순한 int 두 개는 쉽지만, 이런 경우를 생각해보자.

```
write(fd, buffer, size);
```

여기서 buffer는 포인터다. 포인터 값을 그대로 보내면 의미가 없다. 서버에서 그 주소는 전혀 다른 공간을 가리킨다.

그래서 RPC에서는:

- 포인터를 그대로 보내는게 아니라 실제 데이터 내용을 복사해서 보내야 한다.

즉,

```
buffer → "hello world" (데이터 자체를 직렬화해서 전송)
```

이게 바로 복잡한 자료구조에서 RPC가 어려워지는 이유다.

두 번째는 성능과 병행성 문제다.

아주 단순한 서버는 이렇게 동작할 수 있다.

```
while (1) {  
    요청 하나 받음  
    처리  
    응답  
}
```

이건 한 번에 하나만 처리한다.

만약 요청 하나가 오래 걸리면 다른 요청은 전부 대기한다.

그래서 실제 서버는 보통 쓰레드 풀(thread pool)을 쓴다.

구조는 이렇다.

- 메인 쓰레드는 요청만 계속 받는다.
- 요청이 오면 워크 쓰레드에게 넘긴다.
- 여러 요청을 동시에 처리한다.

예를 들어

```
클라이언트 1 → add(2,3)  
클라이언트 2 → add(10,20)  
클라이언트 3 → add(5,7)
```

이 세 개가 동시에 처리될 수 있다.

대신 이 경우에는 공유 자원 접근 때문에 락(Lock)같은 동기화 문제가 생긴다.

세 번째는 네트워크 실패 문제다.

RPC는 내부적으로 네트워크를 쓰기 때문에, 앞에서 배운 문제가 그대로 들어간다.

예를 들어,

- 요청 메시지가 유실될 수 있다.
- 응답이 유실될 수 있다
- 중복 요청이 발생할 수 있다

그래서 실제 RPC 시스템은 다음 같은 것들을 고려해야 한다.

- 타임아웃 + 재전송
- 메시지 ID(중복 제거)
- 정확히 한 번 실행 보장 문제

이 부분이 단순한 함수 호출과 RPC의 가장 큰 차이다.

런타임 라이브러리

여기서 말하는 런타임 라이브러리는 RPC 에서 “진짜 일을 하는 부분”이다. 스텝은 함수 호출을 메시지로 바꿔주는 역할이라면, 런타임은 그 메시지를 실제로 네트워크로 보내고, 실패를 처리하고, 올바른 서버를 찾아가는 전 과정을 책임진다.

즉, RPC 가 단순한 함수 호출처럼 보이게 만드는 핵심 엔진이다.

가장 먼저 현실적인 문제부터 시작한다.

클라이언트는 “어디에 있는 서버를 호출해야 하는가?”를 알아야 한다.

가장 단순한 방식은 다음과 같다.

서버 위치 = (IP 주소, 포트 번호)

예를 들어:

192.168.0.10:8080

클라이언트는 이 주소로 메시지를 보내면 된다.

여기서 포트 번호는 같은 컴퓨터 안에서도 여러 서비스가 동시에 돌 수 있기 때문에 “어떤 프로그램과 통신할지”를 구분해주는 역할을 한다.

좀 더 현실적인 시스템에서는 이런걸 사람이 직접 쓰지 않고 DNS 같은 이름 시스템을 통해 해결한다.

service.example.com → 192.168.0.10

이걸 naming 문제라고 부른다. 분산 시스템에서는 항상 등장하는 기본 문제다.

이제 서버 위치를 알았다고 치자. 그 다음 문제가 더 중요하다.

어떤 네트워크 프로토콜 위에서 RPC 를 만들 것인가?

선택지는 크게 두가지다.

- | |
|--------------------------------------|
| 1. TCP(신뢰할 수 있음)
2. UDP(신뢰할 수 없음) |
|--------------------------------------|

직관적으로는 이렇게 생각하면 된다.

“당연히 TCP 써야지. 메시지 안날아가고 안전하니까” 하지만 실제로는 그렇게 단순하지 않다.

TCP 위에서 RPC 를 만들면 어떤일이 벌어지는지 보자.

RPC 는 기본적으로 이런 흐름이다.

클라이언트 → 요청 → 서버
서버 → 응답 → 클라이언트

그런데 TCP 자체도 신뢰성을 보장하기 위해 내부적으로 이런걸 한다.

메시지 보냄 → ACK 받음
응답 보냄 → ACK 받음

즉, 실제로는 이렇게 된다.

요청 → (ACK)
응답 → (ACK)

결과적으로 RPC 하나를 위해 메시지가 더 많이 오간다.

이건 성능에 직접적인 영향을 준다. 특히 짧고 많은 RPC 호출에서는 매우 비효율적이다.

그래서 실제 많은 RPC 시스템은 오히려 UDP 를 선택한다.

UDP 는 특징이 이렇다.

- 빠름, 단순함, 대신 신뢰성 없음(유실, 중복, 순서 뒤바뀔 가능)

즉, “가볍지만 위험한” 통신 방식이다.

그럼 질문이 생긴다.

“UDP 는 신뢰성이 없는데 RPC 는 어떻게 제대로 동작하나” 이걸 해결하는게 바로 런타임 라이브러리의 핵심 역할이다.

RPC 런타임이 직접 신뢰성을 구현한다.

방법은 이미 앞에서 배운 것들이 그대로 들어간다.

1. 타임아웃 + 재전송

클라이언트가 요청을 보내고 일정 시간 동안 응답이 없으면
“이거 날아갔다고 판단” → 같은 요청 다시 보냄

2. ACK 개념 활용

서버가 응답을 보내면 그 자체가 “응답 + 확인” 역할을 한다.

3. 순서 번호(sequence number)

예를 들어,

요청 1: ID=1, add(2,3)

서버 입장에서 보면, 이미 ID = 1 요청을 처리했음

그래서 서버는 이렇게 한다.

결과만 다시 보냄 (5)

하지만 함수는 다시 실행 안 함

이걸 통해 중복 실행을 방지한다.

이걸 제대로 구현하면 중요한 성질을 만들 수 있다.

- 최소 한 번 실행
- 최대 한 번 실행
- 이상적으로는 정확히 한 번 실행

실제로는 “정확히 한 번” 보장은 어렵고, 보통 “최대 한 번” 정도를 목표로 한다.

다른 문제들

여기서 말하는 내용은 “RPC 가 단순히 요청 -> 응답만 처리하면 끝나는게 아니라, 실제 시스템에서는 훨씬 까다로운 문제들을 계속 만나게 된다”는 이야기다. 하나씩 흐름으로 이어서 보면 이해가 된다.

먼저 가장 현실적인 문제부터 보자. RPC 호출이 오래 걸리는 경우다.

기본 RPC 는 이렇게 동작한다.

```
result = func(x); // 응답 올 때까지 기다림
```

즉, 클라이언트는 요청을 보내고 결과가 올 때까지 멈춘다.

문제는 서버에서 이 작업이 오래 걸릴 때다. 예를 들어,

```
func(x) = 대용량 데이터 처리 (10 초 소요)
```

클라이언트는 타임아웃을 2 초로 설정했다고 해보자. 그러면 어떤 일이 벌어지냐면

1. 클라이언트 요청 보냄
2. 2 초 동안 응답 없음
3. “실패했다” 판단
4. 같은 요청 다시 보냄

하지만 실제로는 서버는 이미 첫 번째 요청을 열심히 처리 중이다.

그래서 결과적으로 같은 요청이 여러 번 실행될 수 있음

이건 매우 위험하다. 특히 결제 같은 작업이라면 큰 문제다. 그래서 해결 방법이 필요하다. 한 가지 방법은 “먼저 받았다는 신호를 보내는 것”이다.

흐름을 보면

클라이언트 → 요청
서버 → "받았음" (ACK)
(시간 경과)
서버 → 최종 결과

이렇게 하면 클라이언트는

- 요청은 도착했구나
- 지금 처리 중이구나

라고 판단하고 불필요한 재전송을 하지 않는다.

더 나아가면 클라이언트가 주기적으로 물어볼 수도 있다.

"아직 처리 중이야?" → 서버: "응, 아직"

이건 긴 작업을 처리할 때 자주 쓰이는 방식이다.

다음 문제는 데이터 크기 문제다.

RPC 호출에 들어가는 인자가 항상 작지는 않다. 예를 들어,

```
func(buffer_100MB)
```

이걸 한 번에 네트워크 패킷에 넣을 수는 없다.

네트워크는 보통 전송 가능한 크기 제한이 있다.

그래서 필요한게

- fragmentation(조개기)
- reassembly(다시 합치기)

예를 들면:

원본 데이터 (100MB)
→ 1MB 씩 나눔 → 100 개 패킷

서버에서는:

100 개 패킷 수신 → 다시 합쳐서 원본 복원

이 기능은 경우에 따라

- 네트워크 계층이 해주기도 하고 아니면 RPC 런타임이 직접 구현해야 한다.

세 번째는 엔디안 문제(byte ordering)다.

이건 꽤 중요하지만 처음엔 잘 안 와닿는다. 같은 숫자라도 컴퓨터마다 저장 방식이 다를 수 있다. 예를 들어, 정수 1 을 4 바이트로 저장한다고 하자.

빅 엔디안:

```
00 00 00 01
```

리틀 엔디안:

```
01 00 00 00
```

이 상태에서 그냥 바이트를 그대로 보내면 어떻게 될까?

- 보낸 쪽: 1 이라고 생각
- 받은 쪽: 전혀 다른 값으로 해석

그래서 RPC 에서는 표 준 포맷을 강제한다. 대표적인 방식이 XDR 이다.

흐름은 이렇게 된다.

클라이언트 (리틀 엔디안)

- XDR 형식으로 변환
- 네트워크 전송
- 서버에서 XDR → 자기 방식으로 변환

즉, 중간에 “공통 언어”를 하나 두는 방식이다.

마지막으로 중요한 문제가 하나 더 있다.

동기 vs 비동기 호출이다. 기본 RPC 는 동기 방식이다.

```
result = func(x); // 여기서 멈춤
```

이건 단순하지만 비효율적일 수 있다. 왜냐하면 기다리는 동안 아무것도 못 하기 때문이다. 그래서 비동기 RPC 가 나온다.

```
handle = func_async(x);  
다른 작업 수행...  
result = get_result(handle);
```

흐름을 보면

1. 요청을 보내고 바로 리턴
2. 클라이언트는 다른 일 수행
3. 나중에 결과 필요할 때 확인

예를 들어:

```
func_async(A)
func_async(B)
func_async(C)
```

이렇게 여러 요청을 동시에 날리고 나중에 한꺼번에 결과를 받을 수도 있다. 이건 성능을 크게 개선할 수 있다.

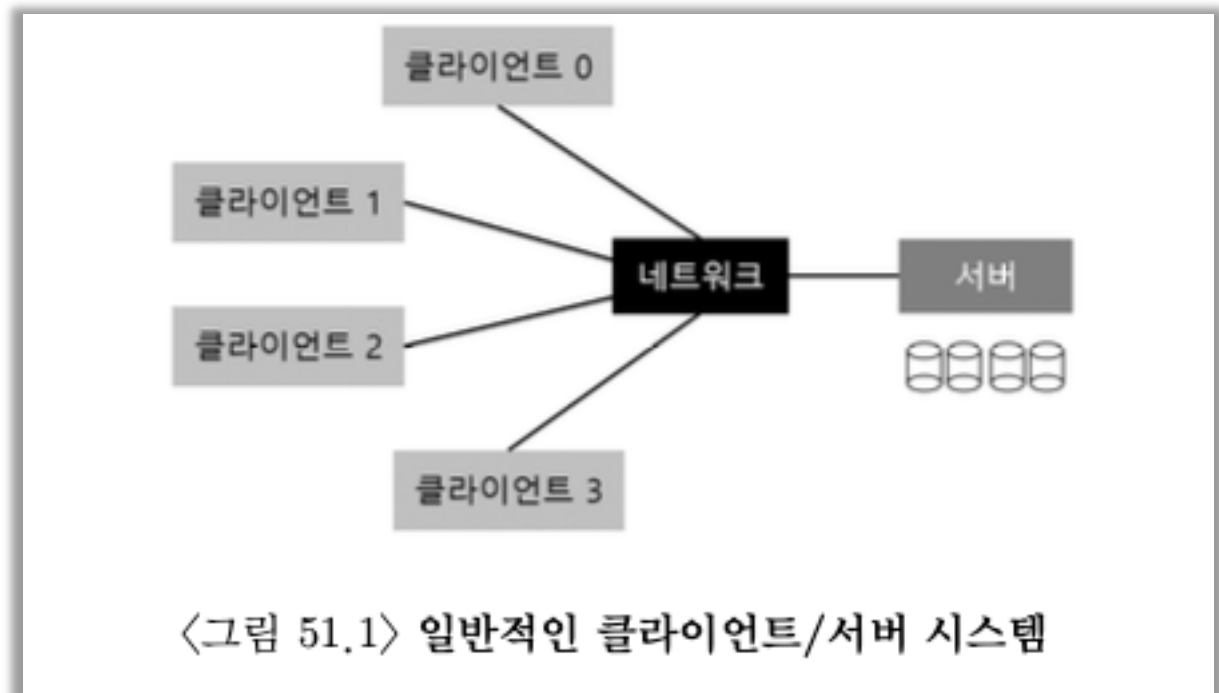
전체를 하나로 묶어서 보면 RPC 런타임이 해결해야 하는 문제는 단순히 “메시지 전송”이 아니다.

- 요청이 오래 걸릴 때 타임아웃과 충돌하지 않게 처리
- 큰 데이터를 쪼개서 보내고 다시 조립
- 서로 다른 시스템 간 데이터 표현 차이 해결
- 동기/비동기 실행 방식 지원

즉, RPC는 “함수 호출처럼 보이게 만든다”는 단순한 목표 뒤에, 실제로는 네트워크, 데이터 표현, 성능, 실패 처리까지 전부 감싸고 있는 복잡한 시스템이다.

이걸 이해하면 RPC가 왜 단순한 API가 아니라 “분산 시스템의 핵심 인프라”인지 감이 잡힌다.

[1-2] Sun 사의 네트워크 파일 시스템(NFS)



핵심 질문: 분산 파일 시스템은 어떻게 만드는가

- 분산 파일 시스템을 어떻게 만드는가?
- 고려해야 할 핵심적인 것들은 무엇이 있는가?
- 어떤 것들이 잘못되기 쉬운가?
- 기존의 시스템에서 무엇을 배울 수가 있는가?

이부분은 “왜 굳이 파일을 네트워크를 통해 서버에 두고, 클라이언트가 원격으로 접근하게 만들었는가”를 이해하는게 핵심이다.

단순히 구조 설명이 아니라, 분산 시스템이 실제로 어떻게 쓰이기 시작했는지를 보여주는 대표적인 사례가 바로 분산 파일 시스템이다.

기본 구조는 단순하다. 클라이언트 여러 대가 있고, 서버는 소수이며, 실제 데이터는 서버 디스크에 저장된다. 클라이언트는 파일을 직접 읽는게 아니라, 네트워크로 요청을 보내서 데이터를 받아온다.

즉, 파일 접근이 “디스크 I/O”가 아니라 “메시지 기반 통신”으로 바뀐다.

예를 들어, 클라이언트가 파일을 읽고 싶다면 내부적으로는 이런 일이 일어난다.

```
read("/home/user/data.txt")
```

이 호출은 로컬 디스크를 읽는 것이 아니라, 다음 과 같은 메시지로 변환된다.

서버에게 요청:

"파일 data.txt 의 offset 0 부터 4KB 읽어줘"

서버는 디스크에서 해당 데이터를 읽고

“여기 데이터 있음”

이렇게 응답을 보낸다. 클라이언트 입장에서는 그냥 파일을 읽은 것처럼 보이지만, 실제로는 네트워크를 통해 서버와 통신한 결과다.

이 구조를 쓰는 이유는 단순히 “중앙에 모아두자”가 아니라, 몇 가지 중요한 문제를 해결하기 위해서다.

첫 번째는 데이터 공유다.

각 클라이언트가 자기 로컬 디스크만 쓴다고 생각해보면 문제가 바로 드러난다.

예를 들어,

클라이언트 A: file.txt 수정

클라이언트 B: 같은 file.txt 필요

로컬 디스크 기반이라면 파일을 따로 복사해야 한다. 복사한 이후에는 버전이 달라질 수 있고, 동기화 문제도 생긴다.

반면 분산 파일 시스템에서는:

모든 클라이언트 → 동일한 서버 파일 접근

즉, 같은 파일을 “하나만 두고” 공유한다.

그래서 A가 수정되면 B는 바로 그 결과를 볼 수 있다. 협업이나 데이터 공유가 훨씬 자연스러워진다.

두 번째는 관리의 단순화다.

파일이 각 클라이언트에 흩어져 있으면 관리가 매우 어렵다.

예를 들어, 백업을 생각해보자.

로컬 디스크 방식이면:

클라이언트 100 대 → 각각 백업 필요

이건 현실적으로 관리가 힘들다.

하지만 서버에만 데이터가 있다면

서버 몇 대만 백업

이걸로 끝난다. 훨씬 단순하고 안정적이다.

세 번째는 보안이다.

데이터를 중앙 서버에 두면 물리적, 논리적 통제가 쉬워진다.

예를 들어,

서버 → 접근 권한 관리

서버 → 물리적으로 보호된 공간에 위치

클라이언트는 네트워크로만 접근하므로, 직접 디스크를 건드릴 수 없다.

데이터 유출이나 무단 접근을 통제하기가 훨씬 수월하다.

하지만 여기서 중요한 점은 이 구조가 단순히 장점만 있는게 아니라는 것이다.

이 구조를 쓰는 순간 새로운 문제가 생긴다.

이제 파일 접근이 “로컬 디스크 읽기”가 아니라

네트워크 요청 → 서버 처리 → 응답

이 과정이 된다. 즉,

- 네트워크 지연이 생긴다
- 서버가 병목이 될 수 있다
- 서버가 죽으면 모든 클라이언트가 영향 받는다

그래서 이후 장에서 계속 나오는 주제들이 자연스럽게 이어진다.

- 캐싱(속도 개선)
- 복제 (서버 장애 대응)
- 일관성 유지(여러 클라이언트가 동시에 수정할 때)

[1] 기본적인 파일 시스템

여기서는 “분산 파일 시스템이 실제로 어떻게 동작하느냐”를 다루고 있다. 핵심 아이디어는 단순하다. 데이터는 서버에 있고, 클라이언트는 네트워크를 통해 그 데이터를 마치 자기 로컬 디스크처럼 사용하는 구조다.

전체 구조를 보면 크게 두 가지 요소가 있다.

하나는 클라이언트 측 파일 시스템, 다른 하나는 서버(파일 서버)다.

클라이언트 입장에서 보면 사용 방식은 완전히 동일하다.

프로그램은 그냥 기존처럼 open(), read(), write(), close()같은 시스템 콜을 호출한다.

중요한 점은, 이 호출이 로컬 디스크가 아니라 “원격 서버”로 이어진다는 사실을 사용자 프로그램은 전혀 모른다는 것이다. 이것을 투명성(transparent)이라고 한다.

즉, 분산 파일 시스템의 목표는 이거다.

로컬 파일 시스템이든, 네트워크 너머 파일이든 구분 없이 동일하게 사용하게 만드는 것

이제 실제 동작 흐름을 보자.

예를 들어, 클라이언트 프로그램이 어떤 파일을 읽는 상황을 생각해보자.

예시: 클라이언트가 read()를 호출하는 경우

1. 사용자가 프로그램에서 read(fd, buffer, size)를 호출한다.
2. 이 호출은 클라이언트 측 파일 시스템으로 전달된다.
3. 클라이언트 파일 시스템은 “이 파일은 서버에 있다”라는 것을 알고 있음
4. 서버에게 메시지를 보낸다.
 - “파일 X의 Y 위치에서 size 만큼 읽어줘”
5. 서버는 디스크에서 해당 블록을 읽는다.
6. 읽은 데이터를 네트워크로 클라이언트에게 보낸다.
7. 클라이언트는 받은 데이터를 buffer 에게 복사한다.
8. read() 호출이 끝난다.

이 과정 전체가 사용자 입장에서는 그냥 “파일 읽기” 한 번으로 보인다. 하지만 내부에서는 네트워크 통신이 포함된 복잡한 과정이 수행된다.

여기서 성능과 관련된 중요한 최적화가 하나 있다. 바로 캐싱(cache)이다.

같은 데이터를 반복해서 읽을 때 매번 서버에 요청하면 너무 느리기 때문에, 클라이언트는 데이터를 로컬에 저장해 둔다.

예시: 같은 블록을 두 번 읽는 경우

처음 read(): 서버에 요청 -> 데이터 받아옴 -> 클라이언트 메모리에 저장

두 번째 read(): 서버에 요청 안함 -> 로컬 캐시에서 바로 읽어서 반환

이 경우 두 번째 읽기는 네트워크를 전혀 사용하지 않는다. 성능이 크게 개선된다.

하지만 여기서 바로 문제가 생긴다. 일관성(consistency) 문제다.

예시: 다른 클라이언트가 데이터를 수정한 경우

상황을 만들어보자.

1. 클라이언트 A가 파일 블록 B를 읽어서 캐시에 저장
2. 클라이언트 B가 같은 블록 B를 수정하고 서버에 write()
3. 클라이언트 A가 다시 read() 호출

이때 클라이언트 A는 어떻게 해야 할까?

- 캐시에 있는 데이터를 그대로 쓰면 -> 오래된 데이터(stale data)
- 서버에서 다시 읽으면 -> 정확하지만 느림

이 문제는 분산 파일 시스템에서 매우 어려운 문제 중 하나다. 단순히 캐시를 쓰면 성능은 좋아지지만, 데이터가 서로 다르게 보일 수 있다.

이 구조를 통해 알 수 있는 핵심은 이거다.

분산 파일 시스템은 단순히 “네트워크로 파일을 읽는다” 수준이 아니라, 다음과 같은 문제들을 동시에 해결해야 한다.

- 로컬처럼 보이게 만드는 투명성
- 네트워크 통신 처리
- 캐시를 통한 성능 최적화
- 여러 클라이언트 간 데이터 일관성 유지

그리고 이 구조에서 반드시 필요한 두 가지 소프트웨어가 있다.

1. 클라이언트 측 파일 시스템
 - 시스템 콜을 받아서
 - 필요하면 서버에 요청을 보내고
 - 결과를 사용자에게 전달
2. 파일 서버
 - 실제 데이터를 디스크에 저장
 - 클라이언트 요청을 처리
 - 읽기/쓰기 요청 수행

이 둘이 협력해서 하나의 파일 시스템처럼 동작한다.

마지막으로, 왜 이런 구조를 쓰는지도 다시 연결해서 보면 이해가 더 잘된다.

- 여러 클라이언트가 같은 데이터를 쉽게 공유할 수 있음
- 중앙 서버에서 백업, 관리, 보안 처리가 쉬움
- 클라이언트는 가볍게 유지 가능

대신 그 대가로

- 네트워크 지연
- 캐시 일관성 문제
- 서버 장애 문제

같은 새로운 어려움이 생긴다.

[2] NFS 에 대하여

여기서는 분산 파일 시스템의 대표적인 사례인 NFS(Network File System)가 왜 성공했는지를 설명하고 있다. 앞에서 본 일반적인 클라이언트/서버 기반 분산 파일 시스템 구조를 실제로 산업에서 널리 사용하게 만든 것이 바로 NFS 다.

NFS 는 Sun Microsystems 가 개발한 시스템인데, 단순히 기술 자체만으로 성공한 것이 아니라, “어떻게 공개했느냐”가 핵심이었다.

당시 많은 시스템들은 내부 구현을 숨기는 폐쇄적인 방식으로 만들어졌다.

즉, 특정 회사의 제품을 쓰면 그 회사의 기술에 종속되는 구조였다. 그런데 NFS 는 완전히 다른 접근을 취했다. 내부 구현을 숨기는 대신, 클라이언트와 서버가 어떻게 통신해야 하는지에 대한 규칙(프로토콜)을 명확하게 정의하고 이를 외부에 공개했다.

이게 바로 오픈 프로토콜(open protocol) 전략이다.

중요한 점은 “어떤 코드를 써야 한다”가 아니라 “어떤 메시지를 주고받아야만 한다”만 정의했다는 것이다.

예를 들어, 클라이언트가 파일을 읽고 싶다면 어떤 형식의 요청 메시지를 보내야 하는지, 서버는 어떤 형식으로 응답해야 하는지만 정해놓은 것이다. 그러면 각 회사는 이 규칙만 지키면 내부 구현은 완전히 자유롭게 할 수 있다.

이 방식의 결과는 굉장히 강력했다. 서로 다른 회사가 만든 시스템이라도 같은 NFS 프로토콜만 따르면 서로 통신이 가능해졌다.

예를 들어, 어떤 회사의 클라이언트가 다른 회사의 서버에 접속해서 파일을 읽고 쓰는 것이 가능해진다. 이건 단순한 기술 문제가 아니라 생태계를 만드는 전략이다.

특정 벤더에 묶이지 않고 다양한 제품이 서로 연결될 수 있기 때문이다.

실제로 이 전략 덕분에 NFS 는 빠르게 확산되었다. 많은 회사들이 자체적으로 NFS 서버를 구현하기 시작했고, 그 결과 시장 자체가 커졌다.

대표적으로 Oracle(Sun 을 인수), NetApp, EMC, IBM 같은 기업들이 NFS 기반 제품을 제공하게 되었다. 중요한 건 특정 하나의 회사가 시장을 독점한 것이 아니라, 여러 회사가 경쟁하면서도 같은 표준을 공유하는 구조가 만들어졌다는 점이다.

이걸 앞에서 배운 내용과 연결해 보면 더 의미가 분명해진다. 분산 파일 시스템은 기본적으로 클라이언트와 서버가 메시지를 주고받는 구조다.

그런데 이 메시지 형식을 공개해버리면, 누구나 그 구조위에 시스템을 만들 수 있다.

결과적으로 기술이 더 빠르게 발전하고, 사용자 입장에서는 선택지가 많아진다.

정리하자면, NFS 의 성공은 단순히 “좋은 분산 파일 시스템이어서”가 아니라, 프로토콜을 공개함으로써 다양한 시스템 간 호환성을 확보하고 시장 장체를 키운 전략 덕분이다.

이부분은 이후 분산 시스템 전반에서 반복되는 중요한 패턴이기도 하다.

REST API, HTTP, TCP/IP 같은 것들은 결국 같은 철학 위에 있다.

[3] 핵심: 단순하고 빠른 서버 크래시 복구

여기서 NFS의 설계 철학 중에서도 가장 중요한 부분 하나를 짚고 있다. 바로 “서버가 죽었을 때 어떻게 빠르게 복구할 것인가”라는 문제다.

특히 초기 버전인 NFSv2는 이 문제를 중심에 두고 설계되었다.

분산 파일 시스템 환경을 다시 떠올려보면 구조는 간단하다.

여러 클라이언트가 하나의 서버에 붙어서 파일을 읽고 쓰는 구조다. 이때 서버는 실제 데이터를 디스크에 저장하고 있기 때문에 사실상 시스템의 중심이다.

그래서 문제가 하나 생긴다.

서버가 죽으면 전체 시스템이 멈춘다. 클라이언트는 여전히 살아있지만, 파일을 읽거나 쓸 대상이 사라졌기 때문에 아무 일도 할 수 없다.

예를 들어보자. 여러 개발자가 하나의 파일 서버를 사용한다고 하자.

- 클라이언트 A: 코드 파일 read()
- 클라이언트 B: 로그 파일 write()
- 클라이언트 C: 디렉터리 탐색

이 모든 요청은 서버를 통해 처리된다. 그런데 서버가 갑자기 다운되면

- A는 파일을 읽을 수 없고
- B는 데이터를 저장할 수 없고
- C는 디렉터리도 볼 수 없다.

즉, 서버 하나의 장애가 전체 시스템 중단으로 이어진다.

그래서 NFSv2를 설계할 때 핵심 질문은 이거였다.

“서버는 반드시 고장난다. 그럼 어떻게 빠르게 다시 살아나게 만들 수 있을까?”

여기서 중요한 설계 선택이 나온다. 바로 서버를 최대한 단순하게 만들자는 것이다.

왜냐하면 서버가 복잡해질수록, 크래시 이후 복구할 때 복구해야 할 상태(state)가 많아진다. 반대로 서버가 단순하면, 그냥 재시작만 해도 바로 정상 동작으로 돌아올 수 있다.

이걸 이해하려면 “상태(state)”라는 개념을 보면된다.

상태가 많은 서버의 문제

만약 서버가 다음과 같은 정보를 계속 들고 있다고 가정해보자.

- 어떤 클라이언트가 어떤 파일을 열었는지
- 파일의 현재 읽기 위치(offset)
- 락(lock) 정보
- 진행 중인 작업 상태

이런 상태들이 메모리에 저장되어 있다가 서버가 죽으면 전부 사라진다. 그러면 서버가 다시 살아났을 때 문제가 생긴다.

예를 들어,

- 클라이언트는 “파일을 이미 열어둔 상태”라고 생각함
- 서버는 “아무것도 모르는 초기 상태”

이 상태와 불일치는 심각한 오류를 만든다. 복구를 하려면 이전 상태를 디스크에 저장해놨다가 다시 읽어와야하는데, 이 과정 자체가 복잡하고 느리다.

그래서 NFSv2 는 반대로 간다.

서버를 “상태가 없는(stateless)” 구조로 만든다.

서버는 다음을 기억하지 않는다.

- 누가 파일을 열었는지
- 이전 요청이 무엇이었는지
- 어떤 작업이 진행 중이었는지

즉, 각 요청을 완전히 독립적인 하나의 작업으로 처리한다.

[4] 빠른 크래시 복구의 열쇠: 상태를 유지하지 않음

여기서는 NFS 설계의 핵심 철학을 한 단계 더 깊게 들어가서 설명한다.

앞에서 “빠른 크래시 복구를 위해 서버를 단순하게 만든다”고 했는데, 그걸 실제로 어떻게 달성했는지를 구체적으로 보여주는 부분이다.

결론부터 말하면 방법은 하나다. 서버가 아무 상태도 기억하지 않도록 만드는 것이다.

우선 “상태”가 정확히 무엇인지 위에서 정의했다. 일반적인 시스템에서 상태라는 것은 어떤 작업이 진행되면서 계속 누적되고 갱신되는 정보다.

이런 정보들을 서버가 기억하고 있으면 그 시스템은 stateful(상태 유지)시스템이다.

이해하기 위해 먼저 우리가 익숙한 로컬 파일 시스템 동작을 보자.

예시: 일반적인 stateful 방식(open/read)

```
int fd = open("foo", O_RDONLY);
read(fd, buffer, MAX);
read(fd, buffer, MAX);
```

이 코드에서 중요한 점은 read()를 두 번 호출했을 때 자동으로 “다음 위치”에서 읽는다는 것이다. 왜 이런일이 가능할까?

그 이유는 시스템이 파일 오프셋(offset)이라는 상태를 내부적으로 유지하고 있기 때문이다.

- 첫 번째 read(): offset = 0 -> MAX 만큼 읽음 -> offset = MAX 로 증가
- 두 번째 read(): offset = MAX -> 그 다음 위치부터 읽음

즉, read() 호출만 봐서는 어디서 읽는지 알 수 없고, 내부 상태(오프셋)를 기반으로 동작한다. 이 상태는 커널이나 서버의 메모리에 저장된다.

이제 이 구조를 그대로 분산 환경(클라이언트-서버)으로 가져온다고 생각해보자.

예시: stateful 분산 파일 시스템

1. 클라이언트 -> 서버: “파일 foo 열어줘”
2. 서버 -> 클라이언트: fd 반환(예: fd=5)
3. 클라이언트 -> 서버: “fd=5 기준으로 읽어줘”

이때 fd는 단순 숫자가 아니라, 서버 내부 상태를 가리키는 핸들이다.

서버는 내부적으로 “fd =5 -> foo 파일, offset= 현재 위치”같은 정보를 가지고 있다.

여기서 문제가 발생한다.

문제 1: 서버 크래시

상황을 보자.

1. 클라이언트가 open(“foo”)호출 -> fd=5 받음
2. read(fd=5) 한 번 수행
3. 서버 크래시 발생
4. 서버 재시작
5. 클라이언트가 다시 read(fd=5)요청

이때 서버입장에서는 fd=5가 무엇인지 전혀 알 수 없다. 왜냐하면

- fd 테이블은 메모리에 있었고,
- 크래시로 인해 전부 날아갔기 때문이다.

즉, 클라이언트는 “계속 이어서 읽고 있다”고 생각하지만, 서버는 “처음 보는 요청”이다.

이 상태 불일치를 해결하려면 복잡한 복구 프로토콜이 필요하다.

예를 들어, 클라이언트가 “fd=5는 foo 파일이다”같은 정보를 다시 알려줘야 한다.

문제 2: 클라이언트 크래시

이번에는 반대 상황이다.

1. 클라이언트가 open(“foo”) 호출
2. 서버는 해당 파일을 열고 상태 유지
3. 클라이언트가 크래시
4. close() 호출이 오지 않음

이때 서버는 어떻게 해야할까?

- 파일을 계속 열린 상태로 둘 것인가?
- 언제 닫아야 하는가?

이걸 해결하려면

- 클라이언트의 생존 여부를 추적하거나
- 타임아웃을 두거나
- 별도의 정리(cleanup) 메커니즘이 필요하다.

즉, 상태를 유지하면 유지할수록 관리해야 할 문제가 기하급수적으로 늘어난다.

그럼 어떻게 동작할까? 핵심은 하나다.

모든 요청에 필요한 정보를 전부 포함시켜서 보낸다.

예시: stateless 방식(NFS 스타일)

클라이언트가 파일을 읽는 경우:

요청 메시지:

- “파일 foo 에서 offset=0 부터 4KB 읽어줘”

다음 요청:

- “파일 foo 에서 offset=4096 부터 4KB 읽어줘”

여기서는 fd 도 없고, 서버가 offset 을 기억하지도 않는다. 클라이언트가 매번 “어디서 읽을지”를 직접 지정한다.

이 방식의 효과는 크래시 상황에서 극적으로 드러난다.

예시: 서버 크래시 후

1. 클라이언트가 요청 보냄 -> 서버 처리 중 크래시
2. 서버 재시작
3. 클라이언트가 같은 요청 다시 보냄

서버 입장에서는 그냥 “새로운 요청 하나”다. 이전 상태를 몰라도 전혀 문제 없다. 그래서 복구 과정이 필요없다. 그냥 다시 처리하면 된다.

이 구조에서 역할이 완전히 바뀐다.

- 서버: 아무것도 기억하지 않음 -> 단순, 빠른 복구
- 클라이언트: 더 많은 책임을 짐 -> 모든 정보 포함해서 요청

즉, 복잡성을 서버에서 클라이언트 쪽으로 밀어낸 것이다.

[5] NFSv2 프로토콜

핵심질문: 어떻게 상태를 유지하지 않는 프로토콜을 정의할까

- 어떻게 상태를 유지하지 않고도 연산이 가능하도록 네트워크 프로토콜을 정의할까?
- 분명한 것은 open()과 같은 상태를 유지하는 호출은 논의에서 제외되어야 한다.(서버는 열려 있는 파일을 관리해야 하기 때문이다.)
- 하지만, 클라이언트 응용 프로그램은 open(), read(), write(), close()와 다른 표준 API 호출들을 사용하여 파일들과 디렉터리들을 접근하고 싶을 것이다.
- 그러므로 질문을 가다듬어보자.
- 상태를 유지하지 않으면서 POSIX 파일 시스템 API 를 지원하는 프로토콜을 정의하는 방법은 무엇인가?

이 부분은 NFSv2 가 실제로, “파일을 어떻게 접근하는지”를 아주 구체적인 메시지 단위로 보여주는 핵심이다. 중요한 건 로컬 파일 시스템처럼 보이지만, 실제로는 모든 동작이 명시적인 메시지 교환으로 이루어진다는 점이다.

먼저 NFS 에서 파일을 직접 “경로”로 계속 다루지 않는다.

대신 파일 핸들(file handle)이라는 것을 사용한다. 이 파일 핸들은 내부적으로 “어떤 파일인지”를 식별할 수 있는 정보 묶음인데, 보통 다음과 같은 요소로 구성된다.

- 어떤 파일 시스템(볼륨)인지
- 그 안에서 어떤 파일인지(아이노드 번호)
- 재사용 방지를 위한 생성 번호

이걸 사용하는 이유는 단순하다. 경로 문자열을 계속 보내는 것보다 훨씬 효율적이고, 서버 입장에서든 빠르게 찾을 수 있기 때문이다.

이제 실제 동작 흐름을 하나의 예로 이어서 보면 이해가 확실히 된다.
클라이언트가 /foo.txt 파일을 읽고 싶다고 해보자.

처음에는 이 파일에 대한 파일 핸들이 없다. 그래서 먼저 LOOKUP 을 해야 한다.
클라이언트는 이미 루트 디렉터리(/)의 파일 핸들을 알고 있다고 가정한다. 그러면 다음과 같은 요청을 서버로 보낸다.

- LOOKUP(루트 핸들, “foo.txt”)

이 요청의 의미는 간단하다. “루트 디렉터리 안에서 foo.txt 라는 파일을 찾아서 그 파일의 핸들을 달라” 서버는 이를 처리한 뒤 다음을 돌려준다.

- foo.txt 의 파일 핸들
- 그리고 파일 속성 정보(크기, 시간 등)

이제 클라이언트는 더 이상 “foo.txt”라는 문자열을 사용할 필요가 없다.
이후 모든 작업은 이 파일 핸들을 기준으로 이루어진다.

그 다음 실제 데이터 읽기 단계로 들어간다.
클라이언트가 파일의 앞부분 100 바이트를 읽고 싶다면 다음과 같은 메시지를 보낸다.

- READ(파일 핸들, offset=0, count=100)

이 요청은 완전히 명시적이다. “이 파일에서 0 번 위치부터 100 바이트 읽어라”

서버는 디스크에서 해당 위치를 읽고, 데이터를 클라이언트에게 반환한다.
여기서 중요한 점은 서버가 아무 상태도 기억하지 않는다는 것이다.

즉, “이 클라이언트가 이전에 어디까지 읽었는지” 전혀 모른다.
그래서 만약에 다음에 이어서 읽고 시다면

- READ(파일 핸들, offset=100, count=100)

이렇게 다시 요청해야 한다. 오프셋을 항상 클라이언트가 직접 넘긴다.

WRITE 도 완전히 동일한 구조다.

예를 들어, 파일의 특정 위치에 데이터를 쓰고 싶다면

- WRITE(파일 핸들, offset=200, count=50, data)

이렇게 보낸다.

서버는 그냥 그 위치에 데이터를 써주고 끝이다.

“이 파일이 열려 있는지”, “누가 쓰고 있는지” 같은 상태는 전혀 관리하지 않는다.

GETATTR 도 중요한 역할을 한다.

- GETATTR(파일 핸들)

이 요청은 해당 파일의 메타데이터를 가져오는 것이다.

예를 들어,

- 파일 크기
- 마지막 수정 시간
- 권한

같은 정보들이 포함된다.

이게 왜 중요하냐면, 클라이언트 캐시와 관련이 있다.

클라이언트가 “내가 가진 데이터가 아직 최신인지” 확인하려면 서버의 수정 시간을 확인해야 하기 때문이다.

이 전체 구조에서 핵심은 딱 하나다.

모든 요청이 완전한 정보(self-contained)를 가지고 있다는 점이다.

- 어떤 파일인지 -> 파일 핸들
- 어디를 읽을지 -> offset
- 얼마나 읽을지 -> count

이렇게 요청 하나만 보면 서버는 바로 처리할 수 있다. 이전 상태를 전혀 기억할 필요가 없다.

이걸 상태 기반 시스템(open/read 방식)과 비교해 보면 차이가 더 명확해진다.

일반적인 로컬 파일 시스템에서는

- open(“foo.txt”) -> fd 반환
- read(fd) -> 내부 offset 자동 증가

이렇게 동작한다.

하지만 NFSv2에서는

- open 개념이 사실상 없음
- 매 read 마다 offset 을 직접 지정

즉, 상태를 서버가 아니라 클라이언트가 책임진다.

이 설계가 중요한 이유는 크래시 상황에서 드러난다.

만약 서버가 갑자기 죽었다가 다시 살아나도

- 파일 핸들 구조는 그대로
- 요청은 항상 독립적

그래서 클라이언트는 그냥 같은 요청을 다시 보내면 된다.

예를 들어:

- READ(handle, 0, 100) 보냈는데 응답 못 받음
- > 서버 재시작
- > 다시 READ(handle, 0, 100)

문제 없이 동일한 결과를 얻는다.

[6] 프로토콜에서 분산 파일 시스템으로

이 부분은 “프로토콜(NFS 메시지)”이 실제로 어떻게 하나의 파일 시스템처럼 동작하게 되는지를 보여주는 핵심이다.

중요한건, 응용 프로그램은 평소처럼 open(), read(), close()를 쓰고 있지만, 그 아래에서는 완전히 다른 방식으로 일이 처리된다는 점이다.

먼저 전체 구조를 보면 역할이 명확하게 나뉜다. 클라이언트 측 파일 시스템은 “로컬 파일 시스템처럼 보이게 만드는 역할”을 하고, 서버는 “요청이 오면 그때그때 처리만 하는 역할”을 한다.

서버는 상태를 기억하지 않기 때문에, 실제로 파일의 상태 관리(파일 위치, 열린 파일 등)는 전부 클라이언트가 담당한다.

이걸 실제 흐름으로 보면 훨씬 이해가 잘된다.

응용 프로그램이 다음 코드를 실행한다고 해보자.

```
fd = open("/foo", ...);  
read(fd, buffer, MAX);  
read(fd, buffer, MAX);  
close(fd);
```

겉으로 보면 그냥 로컬 파일 읽기처럼 보이지만, 내부에서는 꽤 많은 일이 벌어진다.

먼저, open("/foo")가 호출되면, 클라이언트 파일 시스템은 서버에게 바로 “open”을 보내지 않는다. 대신 파일을 찾기 위해 LOOKUP 요청을 보낸다.

클라이언트 -> 서버:

- LOOKUP(루트 디렉터리 핸들, “foo”)

서버는 루트 디렉터리에서 “foo”를 찾고, 그 파일의 파일 핸들(FH)과 속성 정보를 돌려준다.

클라이언트는 이 결과를 받아서 내부 자료구조(열린 파일 테이블)에 다음 정보를 저장한다.

- 파일 디스크립터(fd)
- 해당 파일의 파일 핸들(FH)
- 현재 파일 위치(offset=0)

그리고 나서야 응용 프로그램에게 fd 를 반환한다.

여기서 중요한 포인트는 하나다. 서버는 “이 파일이 열렸다”는 사실을 전혀 모른다.
open 상태는 오직 클라이언트 안에만 존재한다.

이제 read(fd, buffer, MAX)가 호출된다.
클라이언트는 내부 테이블에서 fd 를 찾아서

- 파일 핸들(FH)
- 현재 오프셋(offset=0)

을 가져온다.

그리고 서버로 다음 요청을 보낸다.

- READ(FH, offset=0, count=MAX)

서버는 이 요청을 받으면

- FH -> 어떤 파일인지 확인
- offset -> 어디서부터 읽을지 계산
- 디스크에서 데이터 읽기

그리고 데이터를 클라이언트에게 보낸다.

클라이언트는 데이터를 받으면

- 사용자 버퍼에 복사
- 내부 offset 을 증가(offset=MAX)

이제 다음 read 가 들어오면

- READ(FH, offset=MAX, count=MAX)

이렇게 된다.

세 번째 read 는 offset=2*MAX, 이런식으로 계속 진행된다.

즉, “파일을 순차적으로 읽는다”는 개념은 사실 서버가 아니라 클라이언트가 직접 구현하고 있는 것이다.

마지막으로 close(fd)를 보면 더 흥미롭다.

로컬 파일 시스템에서는 close 를 하면 커널이 파일 상태를 정리하지만, NFS 에서는

- 클라이언트 내부 자료구조만 정리
- 서버에서는 아무 메시지도 보내지 않음

서버는 애초에 “열린 파일”이라는 개념 자체를 모르기 때문이다.

또 하나 중요한 부분은 경로 처리 방식이다.

예를 들어, /home/remzi/foo.txt 같은 긴 경로를 열고 싶다면, 클라이언트는 한 번에 처리하지 못한다. 단계적으로 LOOKUP 을 해야 한다.

1. LOOKUP(“/”, “home”)
2. LOOKUP(“home”, remzi)
3. LOOKUP(“remzi”, “foo.txt”)

이렇게 세 번 요청을 보내서 최종 파일 핸들을 얻는다.

이 구조는 비효율적으로 보일 수 있지만, 대신 설계는 매우 단순해진다.

이 모든 흐름에서 가장 중요한 설계 원칙은 하나다.

서버로 보내는 모든 요청은 “완전한 정보”를 포함한다는 것이다.

예를 들어, READ 요청 하나만 봐도

- 어떤 파일인지 -> FH
- 어디서 읽을지 -> offset
- 얼마나 읽을지 -> count

이 정보만 있으면 서버는 과거 상태 없이도 바로 처리할 수 있다.

이게 중요한 이유는, 크래시 상황에서 차이가 극적으로 드러난다.

서버가 중간에 적었다가 다시 살아났다고 해보자.

클라이언트는 단순히

- 같은 READ 요청을 다시 보내면 된다.

서버 상태를 기억할 필요 없이 다시 처리하면 된다.

반대로 상태를 유지하는 시스템이었다면

- 어떤 파일이 열려 있었는지
- 어디까지 읽었는지

이걸 복구해야 해서 훨씬 복잡해진다.

[7] 서버의 고장을 멱등연산으로 처리하기

이 부분의 핵심은 하나다. “응답이 안 오면 그냥 다시 보내면 되지 않나”라는 단순한 아이디어가 왜 NFS 에서 실제로 통하는지, 그리고 그걸 가능하게 만드는 개념이 바로 멱등성(idempotence)이라는 점이다.

상황부터 자연스럽게 이어서 보자.

클라이언트가 서버에게 어떤 요청을 보냈다고 하자. 그런데 응답이 안온다. 이때 클라이언트 입장에서는 원인을 알 수 없다.

- 요청 자체가 네트워크에서 사라졌을 수도 있고
- 서버는 받았지만 응답이 사라졌을 수도 있고
- 서버가 아예 죽었을 수도 있다.

이 세 가지는 클라이언트 입장에서 구분이 불가능하다. 그래서 NFS 는 복잡하게 따지지 않는다. 그냥 하나로 통일한다.

“응답이 안오면 -> 다시 보낸다”

이걸 가능하게 해주는 메커니즘이 타임아웃/재시도다.

요청을 보내고 일정 시간 기다렸다가 응답이 없으면 같은 요청을 다시 보낸다.

여기서 바로 문제가 하나 생긴다.

“같은 요청을 두 번 보내면 괜찮은가?”

이게 안전하려면 조건이 하나 필요하다.

그 요청이 여러 번 실행돼도 결과가 같아야 한다.

이게 바로 멱등성이다.

개념을 아주 직관적인 예로 보면 이렇다.

메모리의 특정 위치에 값 10 을 저장하는 연산을 생각해보자.

- 한 번 실행 -> 값 = 10
- 세 번 실행 -> 값 = 10

결과가 같다. 이건 멱등이다.

반대로 카운터를 증가시키는 연산은

- 한 번 -> + 1
- 세 번 -> + 3

결과가 달라진다. 이건 멱등이 아니다.

이제 이걸 NFS 연산에 그대로 적용해보면 이해가 확실해진다.
먼저 LOOKUP 과 READ 는 아주 단순하다.
이건 데이터를 “읽기만”하기 때문에 당연히 멍등이다.

- LOOKUP 을 두 번해도 결과는 동일
- READ 를 두 번해도 가온 데이터를 반환

문제가 없다.

흥미로운건 WRITE 다.

쓰기 연산은 일반적으로 멍등이 아니라고 생각하기 쉽다.

그런데 NFS 에서는 WRITE 도 멍등이다. 왜냐하면 WRITE 요청이 이렇게 생겼기 때문이다.

- WRITE(파일 핸들, offset, 길이, 데이터)

핵심은 offset 이 명시적으로 포함되어 있다는 것이다.

예를 들어:

- offset 100 위치에 “ABC”를 써라

이 요청을 한 번 실행하든 두 번 실행하든 결과는 같다.

결국 100 번 위치에는 “ABC”가 들어간다.

즉, “어디에 무엇을 쓸지”가 완전히 지정되어 있기 때문에 반복 실행해도 동일한 결과가 나온다. 그래서 WRITE 도 멍등이 된다.

이제 실제 장애 상황을 예로 보면 이 구조가 얼마나 강력한지 보인다.

케이스 1: 요청이 네트워크에서 사라짐

클라이언트는 응답을 못받는다 -> 다시 WRITE 요청 전송 -> 서버가 실행 -> 정상 완료

케이스 2: 서버가 죽어있다가 살아남

첫 요청은 무시됨 -> 클라이언트 재전송 -> 살아난 서버가 실행 -> 정상 완료

케이스 3: 서버는 처리했지만 응답이 사라짐

서버는 이미 디스크에 데이터를 썼다 -> 클라이언트는 모른다 -> 다시 요청

-> 서버는 같은 WRITE 를 다시 수행

-> 결과는 동일 -> 문제 없음

이 세 가지 경우를 클라이언트는 전부 “그냥 재전송” 하나로 해결한다.

이게 가능한 이유가 바로 모든 핵심 연산이 멍등이기 때문이다.

하지만 모든 연산이 이렇게 깔끔한 건 아니다.
대표적인 문제가 MKDIR 같은 경우다.

예를 들어

1. 클라이언트가 디렉터리 생성 요청
2. 서버는 성공적으로 생성
3. 응답이 유실됨
4. 클라이언트는 실패로 착각 -> 다시 요청

이제 서버입장에서는 이미 디렉터리가 존재한다. 그래서 두 번째 요청에 대해 “실패”를 반환한다.

결과적으로:

- 실제 상태: 디렉터리는 이미 만들어짐
- 클라이언트 인식: 실패

이런 모순이 생긴다.

이건 멍등하게 만들기 어려운 대표적인 사례다.

결론을 흐름으로 정리하면 이렇다.

NFS는 네트워크와 서버가 언제든지 실패할 수 있다는 전제에서 출발한다.

그래서 복잡하게 상황을 구분하지 않고, “응답이 없으면 재전송”이라는 단순한 전략을 사용한다.

이전략이 성립하려면 요청이 여러 번 실행돼도 결과가 같아야 하고, 이를 위해 NFS는 주요 연산들을 멍등하게 설계했다.

그 결과

클라이언트는 모든 실패를 동일하게 처리할 수 있고 서버는 상태 없이도 안전하게 동작하며 크래시와 메시지 손실을 매우 단순한 방식으로 극복할 수 있다.

[8] 성능 개선하기: 클라이언트 측 캐싱

이 부분은 “왜 분산 파일 시스템이 느릴 수밖에 없는지” 그리고 “그걸 어떻게 현실적으로 해결하는지”를 설명하는 구간이다.

핵심은 단순하다. 네트워크는 느리기 때문에 매번 서버에 요청하면 성능이 망가지고, 그래서 클라이언트가 데이터를 들고 있게(캐싱) 만든다.

먼저 문제 상황부터 보면 이해가 쉽다.

분산 파일 시스템에서는 기본적으로 모든 작업이 이렇게 된다.

- read() -> 서버에 요청 -> 네트워크 -> 디스크 -> 다시 네트워크 -> 클라이언트
- write() -> 서버에 요청 -> 네트워크 -> 디스크

이건 로컬 메모리 접근이랑 비교하면 훨씬 느리다. 그래서 그대로 두면 “투명하게는 동작하지만, 엄청 느린 파일 시스템”이 된다.

그래서 등장하는 게 클라이언트 캐싱이다.

클라이언트는 서버에서 한 번 읽어온 데이터를 그냥 버리지 않고, 자기 메모리에 저장해 둔다.

예를 들어, 어떤 프로그램이 같은 파일을 두 번 읽는 상황을 보자.

첫 번째 read:

- 서버에 READ 요청
- 네트워크 왕복
- 디스크 접근
- 데이터 받아옴(비쌈)

두 번째 read:

- 서버에 요청 안함
- 클라이언트 메모리에서 바로 반환(엄청 빠름)

즉, “처음만 느리고 이후는 빠르게”만드는 전략이다.

이건 데이터뿐만 아니라 메타데이터(파일 크기, 수정 시간 등)에도 동일하게 적용된다.

쓰기에서도 비슷한 최적화를 한다. 이게 write buffering 이다.

일반적으로 write()를 호출하면

- 서버에 바로 보내고
- 디스크에 기록될 때까지 기다린다.

이건 느리다.

그래서 NFS 클라이언트는 이렇게 한다.

1. write() 호출
2. 데이터를 서버로 보내지 않고 클라이언트 메모리에 저장
3. write()는 즉시 리턴

그리고 나중에 일정 시점에 서버로 몰아서 전송한다.

이렇게 하면 응용 프로그램 입장에서는 write()가 매우 빠르게 끝난다.

즉, “실제 디스크 쓰기”와 “응용 프로그램의 write 호출”을 분리한 것이다.

여기까지 보면 거의 완벽해 보인다.

- 읽기 -> 캐시 덕분에 빠름
- 쓰기 -> 버퍼링 덕분에 빠름

하지만 여기서 아주 큰 문제가 하나 생긴다.

캐시 일관성(cache consistency) 문제다.

상황을 하나 보자.

클라이언트 A 와 클라이언트 B 가 같은 파일을 사용하고 있다고 하자.

1. A 가 파일을 읽어서 캐시에 저장
2. B 가 같은 파일을 수정(서버에 write)
3. A 가 다시 파일을 읽음

이때 A 는 서버에 가지 않고 자기 캐시를 사용한다. 그럼 A 는 옛날 데이터를 읽게 된다.
즉,

- 서버 데이터: 최신
- A 의 캐시: 오래된 값

이런 불일치가 생긴다.

이게 왜 어려운 문제냐면, 캐시를 쓰는 순간 “속도”는 얻지만 “정확성(일관성)”을 잃을 수 있기 때문이다. 그리고 분산 시스템에서는

- 여러 클라이언트가 동시에 접근하고
- 네트워크 지연이 있고
- 서버 상태도 변할 수 있어서

이걸 완벽하게 맞추는게 매우 어렵다.

[9] 캐시 일관성 문제

캐시 일관성 문제는 분산 파일 시스템에서 성능을 위해 캐싱을 도입하면서 반드시 따라오는 근본적인 문제다.

상황을 보면 직관적으로 이해된다.

어떤 클라이언트 C1 이 파일 F 를 읽어서 자기 로컬 메모리에 캐싱해 둔다. 그런데 다른 클라이언트 C2 가 같은 파일 F 를 수정해서 새로운 버전(F[v2])을 만든다.

이때, 또 다른 클라이언트 C3 가 파일을 읽으면, 최신 버전이 아니라 예전 버전(F[v1])을 읽을 수도 있다. 즉, “누가 언제 수정했는지”가 다른 클라이언트에게 즉시 반영되지 않는 문제가 발생한다.

이 문제는 크게 두 가지로 나뉜다.

첫 번째는 가시성 문제다.

C2 가 파일을 수정했지만, 그 수정 내용이 아직 서버로 보내지지 않고 C2 의 캐시에만 존재하는 상태라면, 다른 클라이언트는 여전히 오래된 데이터를 보게 된다.

즉, 변경이 “언제 다른 사람에게 보이느냐”가 핵심이다.

두 번째는 오래된 캐시 문제다.

이번에는 반대로 서버에는 이미 최신 데이터(F[v2])가 있는데, C1 은 과거에 읽어둔 F[v1]을 계속 캐시에 들고 있어서 그걸 계속 사용하는 상황이다.

즉, 이미 오래된 데이터를 계속 쓰는 문제다.

NFSv2는 이 두 문제를 완벽하게 해결하지는 못하지만, 현실적인 수준에서 타협한 방식으로 처리한다.

첫 번째 문제(갱신 가시성)는 “닫을 때 반영”방식으로 해결한다.

클라이언트가 파일을 수정하고 close()를 호출하는 순간, 그동안 캐시에 쌓아둔 변경 내용을 서버로 전송한다.

그래서 다른 클라이언트가 이후에 그 파일을 open()하면, 최소한 “닫힌 시점 기준”으로는 최신 데이터를 보게 된다. 이것 close-to-open 일관성이라고 한다.

두 번째 문제(오래된 캐시)는 “접근 전에 확인” 방식으로 해결한다.

클라이언트는 파일을 사용할 때 서버에 GETATTR 요청을 보내서 “이 파일이 마지막으로 언제 수정됐는지”를 확인한다.

만약 서버의 수정 시간이 내가 캐싱한 시점보다 더 최신이라면 내 캐시는 버리고 서버에서 다시 읽어온다. 반대로 변경이 없다면 캐시를 그대로 써서 성능을 확보한다.

그런데 이 방식도 문제가 있다.

매번 파일을 열 때마다 서버에 GETATTR 을 보내면, 실제로는 아무도 파일을 수정하지 않았는데도 계속 서버에 “이거 바뀌었어?”라고 묻는 꼴이 된다.

이건 불필요한 네트워크 트래픽을 만든다.

그래서 등장한 게 속성 캐시(attribute cache)다. 클라이언트는 파일 속성 정보(수정 시간 등)를 일정 시간(예: 3 초) 동안 캐시에 저장해 두고, 그 시간 동안은 서버에 물어보지 않고 그냥 사용한다. 이 시간 안에서는 약간 오래된 데이터를 볼 가능성을 감수하는 대신, 성능을 크게 개선한다.

결국 핵심은 이거다. 분산 시스템에서는 “항상 최신 데이터를 보장”하려면 성능이 크게 떨어지고, “성능을 높이려면” 어느정도 오래된 데이터를 허용해야 한다.

NFSv2는 이 둘 사이에서 현실적인 균형을 선택한 설계라고 보면 된다.

[10] NFS의 캐시 일관성 기법에 대한 평가

NFS의 캐시 일관성 기법은 현실적인 타협이란 점에서 의미는 있지만, 실제로는 꽤 애매하고 불편한 특성들을 만든다. 먼저 “닫을 때 서버로 반영(flush-on-close)” 방식은 논리적으로는 맞지만 성능 측면에서는 비효율적인 상황을 만든다.

예를 들어, 어떤 클라이언트가 파일을 잠깐 생성해서 조금 쓰고 바로 삭제하는 경우를 생각해보면, 이 파일은 사실 서버까지 갈 필요가 없다.

메모리에서만 처리하고 끝내는 게 훨씬 효율적이다.

그런데 NFS는 파일을 닫는 순간 무조건 서버로 내용을 보내기 때문에, 이런 짧은 생명주기의 파일조차 네트워크를 타고 서버까지 전달된다.

결과적으로 불필요한 통신이 발생하고 성능이 떨어진다.

더 근본적인 문제는 속성 캐시(attribute cache) 때문에 생기는 일관성의 불확실성이다. 클라이언트는 파일을 사용할 때마다 서버에 물어보는 대신, 일정 시간 동안은 캐시된 속성 정보를 믿고 행동한다.

이 덕분에 성능은 좋아지지만, 그 대가로 “내가 지금 보고 있는 데이터가 최신인지 아닌지”를 확신할 수 없게 된다.

어떤 순간에는 최신 데이터를 읽고, 어떤 순간에는 이미 다른 클라이언트가 수정한 뒤의 이전 데이터를 읽게 된다. 이 차이는 단순히 캐시 타이머가 만료됐는지 여부에 따라 달라지는데, 이건 사용자 입장에서 매우 직관적이지 않다.

문제는 이런 비일관성이 항상 드러나는게 아니라 “가끔” 발생한다는 점이다.

대부분의 경우에는 잘 동작하기 때문에 시스템이 정상처럼 보이지만, 특정 타이밍에서는 오래된 데이터를 읽거나 예상과 다른 결과가 나오게 된다.

이런 종류의 오류는 재현하기 어렵고 디버깅도 까다롭다. 그래서 실제 시스템에서 간헐적인 오작동의 원인이 되기 쉽다.

결국 NFS의 캐싱 방식은 전통적인 파일 시스템처럼 “항상 일관된 결과를 제공한다”는 의미의 시맨틱을 먼저 정하고 구현한 것이 아니라, “성능을 위해 이런 식으로 구현했더니 결과적으로 이런 시맨틱이된다”는 식으로 만들어진 것이다.

즉, 보통은 시맨틱이 구현을 결정하지만, 여기서는 구현상의 선택이 사용자에게 보이는 동작 방식 자체를 결정해버린 사례라고 볼 수 있다.

[11] 서버 측 쓰기 버퍼링의 의미

지금까지 클라이언트 쪽 캐싱이 문제의 중심이었다면, 여기서는 서버 쪽에서의 쓰기 처리 방식이 왜 그렇게 엄격하게 설계되어 있는지를 설명하는 부분이다.

핵심은 하나다.

서버는 “쓰기 요청에 성공했다고 응답하기 전에 반드시 디스크에 실제로 기록을 끝내야 한다”는 점이다.

겉보기에는 서버도 클라이언트처럼 메모리에 데이터를 잠깐 저장해두고 나중에 디스크에 쓰면 더 빠를 것처럼 보인다. 실제로 읽기의 경우는 그렇게 해도 문제가 없다.

디스크에서 읽어온 데이터를 메모리에 캐싱해두면 다음 요청은 빠르게 처리할 수 있기 때문이다. 하지만 쓰기에서는 상황이 완전히 달라진다.

문제가 되는 상황을 보면 이해가 쉽다.

클라이언트가 어떤 파일에 대해 순서대로 세 번 write를 한다. 첫 번째는 a, 두 번째는 b, 세 번째는 c를 쓰는 방식이다.

정상이라면 파일은 a, b, c 순서로 깔끔하게 덮어써져야 한다.

그런데 서버가 두 번째 쓰기(b)를 디스크에 실제로 기록하기 전에, 단지 메모리에만 저장해두고 “성공했다”고 응답했다고 가정해보자.

그 직후 서버가 크래시가 나면, 메모리에 있던 b는 사라진다.

이후 서버가 재시작되고 세 번째 쓰기(c)는 정상적으로 처리된다.

결과적으로 파일 상태는 a, (옛날 값), c 같은 식으로 뒤죽박죽이 된다.

클라이언트 입장에서는 모든 write 가 성공했다고 들었기 완전히 잘못된 상태를 보게 되는 것이다. 이건 단순한 성능 문제가 아니라 데이터 무결성이 깨지는 심각한 문제다.

이 상황이 왜 특히 중요한지 보면, NFS 클라이언트는 기본적으로 “서버가 성공이라는 응답했으면 그 작업은 실제로 완료됐다”고 믿고 동작한다.

그리고 실패하면 재시도하는 방식으로 신뢰성을 확보한다. 그런데 서버가 실제로는 디스크에 쓰지 않았는데 성공이라고 응답해버리면, 클라이언트의 이 가정 자체가 깨진다.

그러면 재시도 메커니즘도 의미가 없어지고, 결국 데이터가 망가진다.

그래서 NFS 서버는 반드시 각 WRITE 요청을 안정적인 저장장치, 즉 디스크 같은 영속 저장소에 완전히 기록한 뒤에야 성공 응답을 보낸다.

이렇게 해야 클라이언트는 “성공 응답 = 실제로 저장 완료”라고 믿을 수 있고, 만약 중간에 서버가 죽으면 클라이언트가 재시도를 통해 문제를 복구할 수 있다.

문제는 이 방식이 성능을 크게 떨어뜨린다는 점이다. 디스크에 쓰는 작업은 느리기 때문에, 매번 write 요청마다 디스크까지 동기적으로 기다리면 병목이 생길 수 밖에 없다.

그래서 실제 시스템에서는 이 문제를 완화하기 위한 다양한 방법이 등장한다.

예를 들어, 배터리로 보호되는 메모리를 사용해서 전원이 나가도 데이터가 유지되도록 한 뒤, 디스크에 쓰지 않고도 빠르게 응답하는 방법이 있다.

또는 쓰기 성능에 최적화된 파일 시스템을 설계해서 디스크 기록 자체를 빠르게 만드는 방식도 사용한다.

결국 여기서의 핵심은 명확하다.

성능을 위해 쓰기를 느슨하게 처리하면 데이터가 깨질 수 있고, 데이터의 정확성을 보장하려면 반드시 디스크까지 확실하게 기록해야 한다. NFS는 후자를 선택했고, 그 대가로 성능 문제를 다른 방식으로 보완하려고 한 것이다.

[1-3] Andrew 파일 시스템(AFS)

andrew 파일 시스템(AFS)은 1980년대 Carnegie Mellon University에서 Satyanarayanan 교수를 중심으로 개발된 분산 파일 시스템으로, 가장 중요한 목표는 확장성이었다.

즉, 가능한 한 많은 클라이언트를 동시에 지원할 수 있도록 설계하는 것이 핵심이었다.

이를 위해 특히 클라이언트와 서버 사이의 통신 방식이 중요하게 다뤄졌다.

기존 NFS 방식에서는 클라이언트가 캐시도니 데이터가 최신인지 확인하기 위해 주기적으로 서버에 질의를 보내야 했고, 이 과정에서 서버의 자원 사용이 증가하여 확장성에 한계가 생겼다. 반면 AFS는 이러한 문제를 줄이면서도 사용자 입장에서 더 이해하기 쉬운 일관성 모델을 제공하는데 초점을 맞췄다.

그 결과, AFS는 파일을 open 할 때 항상 최신 버전을 보장하는 방식으로 캐시 일관성을 단순하고 명확하게 만든 것이 특징이다.

[1] AFS 버전 1

AFS의 첫 번째 버전(AFSv1)은 기본적인 구조는 잘 갖추고 있었지만 확장성 측면에서는 한계가 있었고, 이후 재설계를 통해 AFSv2로 발전하게 된다.

이 초기 버전에서 가장 중요한 설계 원칙은 “파일 전체를 클라이언트 로컬 디스크에 캐싱한다”는 점이다.

동작 방식을 보면 흐름이 명확하다. 클라이언트에서 open()을 호출하면 AFS

클라이언트(Venus)가 서버(Vice)에게 Fetch 요청을 보내고, 서버는 해당 파일의 전체 내용을 클라이언트로 전송한다.

클라이언트는 이 파일을 자신의 로컬 디스크에 그대로 저장한다.

이후 발생하는 read()나 write() 요청은 모두 이 로컬 파일을 대상으로 수행되기 때문에 네트워크 통신이 전혀 발생하지 않고 매우 빠르게 처리된다.

즉, 파일을 한 번 받아오면 그 이후 작업은 사실상 로컬 파일 시스템처럼 동작한다.

작업이 끝나고 close()가 호출되면 상황이 다시 바뀐다. 클라이언트는 파일이 수정되었는지를 확인하고, 변경이 있었다면 Store 요청을 통해 전체 파일을 다시 서버로 전송한다.

이때도 부분이 아니라 파일 전체가 전송된다는 점이 특징이다.

이 구조는 NFS와 비교하면 차이가 분명하다. NFS는 파일을 블록 단위로 나누어 메모리에 캐싱하고 필요할 때마다 서버와 통신하지만, AFSv1은 아예 전체 파일을 한 번에 가져와 로컬 디스크에 저장하고 이후에는 서버와 거의 통신하지 않는다.

대신 파일을 열 때와 닫을 때만 네트워크를 사용한다.

또 하나의 특징은 이미 캐싱된 파일을 다시 사용할 때의 처리 방식이다.

클라이언트는 서버에 TestAuth 요청을 보내 파일이 변경되었는지를 확인한다. 만약 변경되지 않았다면 로컬에 있는 파일을 그대로 사용하고, 변경되었다면 다시 Fetch를 통해 전체 파일을 받아온다. 이 방식은 불필요한 데이터 전송을 줄여 성능을 개선하는 데 도움이 된다.

정리하면, AFSv1 은 “파일 단위 전체 캐싱 + open 시 가져오기 + close 시 반영”이라는 단순한 구조를 통해 성능을 높이려 했고, 이후 버전에서는 이 구조를 기반으로 확장성과 효율성을 더 개선하게 된다.

[2] AFS 버전 1 의 문제점

AFSv1 은 “파일 전체를 캐싱한다”는 단순하고 강력한 아이디어 덕분에 성능 면에는 장점이 있었지만, 실제로 운영해보니 확장성을 제한하는 몇 가지 뚜렷한 문제가 드러났다. 이 문제들은 단순한 추측이 아니라, 실제 시스템을 측정하고 분석한 결과에서 나온 것이라는 점이 중요하다. 이런 측정을 통해 어디서 시간이 낭비되는지, 무엇이 병목인지가 명확해졌다.

첫 번째로 드러난 문제는 “경로명을 따라가는 비용”이다.

AFSv1 에서는 파일링 요청할 때마다 전체 경로명을 서버에 전달한다.

예를 들어, /home/remzi/notes.txt 같은 경로를 넘기면, 서버는 루트 디렉터리부터 시작해서 home 을 찾고, 그 안에서 remzi 를 찾고, 다시 그 안에서 파일을 찾는 식으로 단계적으로 내려가야 한다. 이 작업 자체는 하나만 보면 별거 아닌 것 같지만, 수 많은 클라이언트가 동시에 요청을 보내면 상황이 달라진다.

서버의 CPU 시간이 실제 데이터 처리보다 “경로 탐색”에 대부분 쓰이게 되는 병목이 발생한다.

두 번째 문제는 TestAuth 요청의 과도한 사용이다.

AFSv1 에서는 클라이언트가 자신이 가진 캐시가 여전히 유효한지 확인하기 위해 서버에 계속 물어본다.

이 방식은 NFS 에서 GETATTR 요청이 많아지는 문제와 거의 동일한 구조다.

대부분의 경우 서버는 이에 응답하느라 자원을 소모한다. 결과적으로 서버는 실제 데이터를 처리하는 대신 “변경 여부 확인 응답”에 시간을 많이 쓰게 된다.

이 두 가지가 핵심적인 프로토콜 수준의 문제였다.

추가적으로 시스템적인 문제도 있었다. 서버가 클라이언트마다 별도의 프로세스를 할당하는 구조였기 때문에 문맥 교환 비용이 커졌고, 서버 간의 부하 분산도 효율적이지 못했다.

이를 개선하기 위해 볼륨(volume) 개념을 도입해 데이터를 서버 간에 이동하게 했고, 이후 AFSv2 에서는 프로세스 대신 쓰레드를 사용해 오버헤드를 줄였다.

정리하면 AFSv1 의 문제는 단순히 “느리다”가 아니라, 구조적으로 서버가 불필요한 일을 너무 많이 하도록 설계되어 있었다는 점이다. 경로 탐색과 상태 확인 요청이 서버 자원을 잡아먹으면서, 클라이언트 수가 늘어날수록 시스템이 잘 확장되지 않는 한계를 드러냈다.

[3] 프로토콜의 개선

핵심 질문: 어떻게 확장 가능한 파일 프로토콜을 설계할까

- 서버와의 상호작용의 수를 최소화하려면 프로토콜을 어떻게 재설계해야할까?
- 즉, TestAuth 메시지의 수를 어떻게 하면 줄일 수 있을까?
- 더 나아가, 서버와의 상호작용을 효율적으로 만들려면 프로토콜을 어떻게 설계해야 할까?
- 이 두 개의 문제들을 동시에 대처할 수 있다면 AFS를 확장 가능한 버전으로 만들 수 있다.

앞서 언급한 두 문제는 AFS의 확장성을 제한하였다.

서버 CPU가 시스템의 병목이 되었으며 각 서버는 과부하 문제로 인해 20개의 클라이언트만 서비스할 수 있었다. 서버들은 TestAuth 메시지를 너무 많이 받고 있었으며, Fetch 또는 Store 메시지들을 받으면 디렉터리를 구분하여 경로명을 따라가는 데에 너무 많은 시간을 쓰고 있었다. AFS 설계자들은 위와 같은 문제에 직면하였다.

[4] AFS 버전 2

AFS 버전 2는 확장성을 개선하기 위해 클라이언트와 서버 간의 상호작용 방식을 근본적으로 바꾼 설계가 핵심이다. 가장 중요한 변화는 콜백(callback) 메커니즘의 도입이다.

이는 서버가 “해당 파일이 변경되면 알려주겠다”라고 약속한 방식으로, 클라이언트가 더 이상 주기적으로 서버에 파일 변경 여부를 확인하지 않아도 되게 만든다.

즉, 기존처럼 계속 서버에 묻는 폴링 방식이 아니라, 서버가 필요할 때만 알려주는 구조로 바뀌면서 불필요한 네트워크 트래픽과 서버 부하를 크게 줄였다.

이로 인해 많은 클라이언트를 동시에 처리할 수 있는 확장성이 확보된다.

또 하나의 중요한 변화는 파일을 식별하는 방식이다.

AFSv2에서는 전체 경로명을 매번 서버에 전달했지만, AFSv1에서는 파일 식별자(FID)를 사용한다. 이 식별자는 볼륨 ID, 파일 ID, 그리고 재사용을 구분하기 위한 고유 값으로 구성된다. 클라이언트는 경로를 한 단계씩 따라가며 각 디렉터리 정보를 로컬에 캐싱하고, 이후에는 이 캐시된 정보를 활용한다.

따라서 같은 파일을 다시 접근할 때는 경로를 다시 탐색할 필요 없이 훨씬 빠르게 처리할 수 있다.

파일 접근 과정도 효율적으로 바뀌었다. 클라이언트가 특정 파일을 처음 열 때는 디렉터리와 파일을 순차적으로 서버에서 가져오고 각각에 대해 콜백을 설정한다.

이후에는 해당 파일과 관련된 디렉터리들이 변경되지 않는 한, 모든 접근이 로컬 디스크에서 이루어진다. read나 write 같은 연산도 서버와 통신하지 않고 로컬에서 처리되므로 성능이 크게 향상된다.

파일이 수정된 경우에만 close 시점에 서버로 변경 내용을 전송한다.

결과적으로 AFSv2는 “처음 접근은 느리지만 이후는 매우 빠름”구조를 만든다. 특히 한 번 접근한 파일을 반복해서 사용하는 환경에서는 로컬 파일 시스템과 거의 동일한 성능을

제공한다. 동시에 콜백 기반 일관성 유지 덕분에 불필요한 확인 요청이 사라지면서 서버의 부담이 줄어들고, 이는 전체 시스템의 확장성 향상으로 이어진다.

[5] 캐시 일관성

AFS의 캐싱 일관성은 NFS에서 문제였던 갱신 가시성과 오래된 캐시 문제를 보다 직관적인 방식으로 해결하려는 접근이다. 핵심은 콜백과 전체 파일 캐싱을 결합한 구조에 있다.

AFS에서는 파일의 변경이 서버에 반영되는 시점과 다른 클라이언트들의 캐시가 무효화되는 시점이 명확하게 일치한다.

그 기준은 바로 파일이 닫히는 순간이다. 어떤 클라이언트가 파일을 열어 수정한 뒤 `close()`를 호출하면, 그때 전체 파일이 서버로 전송되고 서버는 해당 파일을 캐시하고 있던 다른 클라이언트들의 콜백을 모두 무효화한다.

이로 인해 다른 클라이언트들은 더 이상 기존 캐시를 사용할 수 없고, 다시 파일을 열 때 반드시 서버로부터 최신 버전을 가져오게 된다. 이 구조 덕분에 “언제 최신 데이터가 보이는가”라는 문제는 파일 `close` 시점으로 명확하게 정의된다.

같은 기계 내부에서는 조금 다른 방식이 적용된다. 동일한 클라이언트 머신 내의 여러 프로세스들은 일반적인 Unix 파일 시스템처럼 동작한다.

즉, 한 프로세스가 파일을 수정하면 `close()`를 하지 않아도 다른 로컬 프로세스는 그 변경 내용을 즉시 볼 수 있다. 이는 로컬 환경에서 기대하는 자연스러운 동작을 유지하기 위한 설계이며, AFS의 일관성 정책은 주로 서로 다른 머신 간의 데이터 일관성 문제를 해결하는데 초점이 맞춰져 있다.

여러 클라이언트가 동시에 같은 파일을 수정하는 상황에서는 AFS는 “마지막 기록자가 승리” 정책을 사용한다.

정확히는 마지막으로 `close()`를 수행한 클라이언트의 파일 내용이 최종 결과가 된다.

각 클라이언트는 파일 전체를 캐싱하고 수정하기 때문에, 최종적으로 서버에 반영되는 것도 전체 파일 단위이다. 따라서 충돌이 발생하면 두 클라이언트의 변경 내용이 섞이는 것이 아니라, 하나의 완전한 파일이 선택된다.

이 점은 블록 단위로 데이터를 갱신하는 NFS와 중요한 차이를 만든다.

NFS에서는 서로 다른 클라이언트의 수정이 파일의 서로 다른 블록에 반영되면서 결과 파일이 뒤섞일 수 있고, 경우에 따라서는 구조적으로 깨진 데이터가 생성될 수 있다.

결과적으로 AFS의 캐시 일관성 모델은 “파일 단위, `close` 시점 기준, 콜백 기반 무효화”라는 명확한 규칙을 갖는다. 덕분에 동작 방식이 비교적 단순하고 예측 가능하며, 특히 파일을 단위로 작업하는 환경에서는 일관성과 성능을 동시에 확보할 수 있다.

[6] 크래시 복구

AFS의 크래시 복구는 NFS보다 훨씬 복잡해지는데, 그 이유는 AFS가 콜백 기반의 상태 정보를 활용하기 때문이다. 즉, 어떤 클라이언트가 어떤 파일을 캐싱하고 있는지에 대한 정보가 시스템 동작에 중요한 역할을 하기 때문에, 이 정보가 깨지는 상황(크래시)이 발생하면 복구 과정이 단순하지 않다.

먼저 클라이언트가 크래시되는 경우를 보면, 문제는 클라이언트가 서버로부터 받아야 할 콜백 무효화 메시지를 놓칠 수 있다는 점이다.

예를 들어, 클라이언트 C1이 파일 F를 캐시에 가지고 있는 동안 재부팅이 발생하고, 그 사이 다른 클라이언트 C2가 파일을 수정하면 서버는 C1에게 캐시 무효화를 통보해야 한다. 하지만 C1은 이 메시지를 받지 못했기 때문에 오래된 데이터를 계속 사용할 위험이 생긴다. 이를 해결하기 위해 클라이언트는 재부팅 이후 자신이 가진 캐시를 그대로 신뢰하지 않고, 파일을 다시 접근할 때마다 서버에 해당 캐시가 유효한지 확인한다. 유효하다면 그대로 사용하고, 그렇지 않다면 최신 데이터를 다시 가져온다.

서버가 크래시되는 경우는 더 복잡하다.

AFS 서버는 어떤 클라이언트가 어떤 파일을 캐싱하고 있는지에 대한 콜백 정보를 메모리에에만 유지한다. 따라서 서버가 다운되었다가 재시작하면 이 정보가 모두 사라진다. 결과적으로 서버는 더 이상 어떤 클라이언트에게 캐시 무효화 메시지를 보내야 하는지 알 수 없게 된다. 이 상태에서 클라이언트들이 기존 캐시를 계속 사용하면 데이터 일관성이 깨질 수 있다.

이 문제를 해결하기 위해서는 서버가 재시작되었음을 클라이언트들이 인지하고, 각자 캐시의 유효성을 다시 확인하도록 만들어야 한다.

즉, 클라이언트는 서버 장애 이후에는 자신이 가진 데이터를 자동으로 신뢰하지 않고, 필요 시 서버에 확인하거나 다시 가져오는 방식으로 동작한다.

이를 구현하기 위한 방법으로는 서버가 재시작 시 모든 클라이언트에게 “기존 캐시는 신뢰하지 말라”는 메시지를 보내는 방식이나, 클라이언트가 주기적으로 서버의 상태를 확인하는 heartbeat 방식 등이 사용된다.

결국 AFS의 크래시 복구는 “상태 기반 시스템의 복잡성”을 그대로 보여준다.

콜백을 통해 성능과 확장성을 얻는 대신, 장애 상황에서는 클라이언트와 서버가 서로의 상태를 다시 맞추는 추가적인 절차가 필요해진다.

[7] AFSv2 의 확장성과 성능

AFSv2 는 프로토콜 수준에서의 변화(콜백, 전체 파일 캐싱 등)를 통해 성능과 확장성 모두에서 AFSv1 보다 크게 개선된 모습을 보인다. 가장 중요한 특징은 서버의 부담을 줄이고, 클라이언트가 최대한 로컬에서 작업하도록 만든 점이다.

실제로 하나의 서버가 훨씬 더 많은 클라이언트를 처리할 수 있게 되었고, 대부분의 작업이 네트워크가 아닌 로컬 디스크나 메모리에서 수행되기 때문에 체감 성능도 로컬 파일 시스템에 가깝게 나타난다.

성능을 구체적으로 보면, 파일을 처음 읽는 경우에는 AFS 와 NFS 모두 비슷한 성능을 보인다.

이는 결국 네트워크를 통해 데이터를 가져오는 비용이 지배적이기 때문이다. 하지만 한 번 읽은 이후의 동작에서 차이가 드러난다. 특히 큰 파일을 다시 읽은 경우 AFS 는 로컬 디스크 캐시에 전체 파일이 저장되어 있기 때문에 디스크 접근만으로 처리할 수 있는 반면, NFS 는 메모리에 일부 블록만 캐시하기 때문에 다시 네트워크를 통해 데이터를 가져와야 한다. 이 경우 네트워크 접근이 디스크보다 느리다면 AFS 가 훨씬 유리해진다.

쓰기 성능도 흥미롭다.

새로운 파일을 생성하거나 순차적으로 쓰는 경우에는 AFS 와 NFS 모두 유사한 성능을 보인다. AFS 는 로컬 디스크에 먼저 쓰고 close 시점에 서버로 전송하며, NFS 는 메모리 버퍼링을 활용하다가 역시 close 시점에 서버로 전달된다.

두 경우 모두 운영체제의 캐시 덕분에 실제 디스크 쓰기 비용이 어느 정도 숨겨지기 때문에 성능 차이가 크지 않다.

하지만 AFS 의 약점도 분명하다. 이미 존재하는 큰 파일을 덮어쓰는 경우 성능이 크게 떨어진다.

AFS 는 파일 전체를 먼저 클라이언트로 가져온 뒤 수정하고 다시 전체를 서버로 보내야 하기 때문이다. 반면, NFS 는 필요한 블록만 덮어쓰면 되므로 훨씬 효율적이다. 또한, 큰 파일 중 일부만 읽거나 쓰는 경우에도 NFS 가 유리하다. AFS 는 파일 전체를 가져오는 구조이기 때문에 작은 작업에도 큰 오버헤드가 발생할 수 있다.

결과적으로 AFS 와 NFS 는 설계 목표가 다르기 때문에 성능 특성도 다르게 나타난다.

AFS 는 반복 접근이 많은 환경, 특히 동일한 파일을 여러 번 읽거나 사용하는 워크로드에서 강력한 성능과 확장성을 제공한다.

반면 NFS 는 부분적인 데이터 접근이나 파일의 일부만 자주 수정하는 환경에서 더 효율적이다. 어떤 시스템이 더 좋은지는 절대적인 기준이 아니라, 실제 사용 패턴에 따라 달라진다.

[8] AFS: 그 외의 개선점들

AFS 는 단순히 성능과 확장성만 개선한 것이 아니라, 실제 사용성과 관리 편의성 측면에서도 중요한 기능들을 추가했다는 점이 특징이다.

먼저 가장 눈에 띄는 변화는 전역적 이름 공간(global namespace)의 제공이다.

AFS 에서는 모든 클라이언트가 동일한 파일을 동일한 경로 이름으로 접근할 수 있도록 보장한다. 즉, 어떤 파일이든 시스템 전체에서 하나의 일관된 이름을 가지며, 사용자 입장에서는 어디서 접속하든 동일한 구조를 보게 된다.

반면, NFS 는 각 클라이언트가 개별적으로 서버를 마운트하는 방식이기 때문에, 같은 파일이라도 클라이언트마다 경로가 다를 수 있고 이를 맞추기 위해 관리자가 별도의 규칙을 강제해야 하는 번거로움이 있다.

보안 측면에서도 AFS 는 훨씬 강화된 기능을 제공한다. 사용자 인증 메커니즘을 포함하고 있어, 특정 파일이나 디렉터리를 해당 사용자만 접근할 수 있도록 설정할 수 있다.

이는 단순한 파일 권한을 넘어서 체계적인 보안 모델을 제공하는 것으로, 여러 사용자가 동시에 사용하는 환경에서 중요한 역할을 한다.

반면, 초기 NFS 는 기본적인 수준의 보안만 제공했기 때문에, 신뢰할 수 있는 내부 네트워크 환경을 전제로 사용하는 경우가 많았다.

접근 제어 역시 AFS 의 중요한 장점이다. 사용자는 어떤 사용자나 그룹이 자신의 파일에 접근할 수 있는지를 보다 유연하게 설정할 수 있다. 이는 전통적인 Unix 파일 시스템의 제한적인 권한 모델보다 더 세밀한 제어를 가능하게 하며, 협업 환경에서 특히 유용하다. NFS 는 이러한 세밀한 공유 제어 기능이 상대적으로 부족하다.

마지막으로 시스템 관리 측면에서도 AFS 는 많은 개선이 이루어졌다. 서버 자원을 효율적으로 관리하고, 데이터의 배치나 이동을 유연하게 할 수 있는 도구들이 제공된다. 예를 들어, 볼륨 단위로 데이터를 이동하거나 재배포할 수 있어 부하 분산이 용이하며, 전체 시스템을 운영하는 관리자 입장에서 훨씬 효율적인 관리가 가능하다.

이런 점에서 AFS 는 단순한 파일 시스템을 넘어, 대규모 분산 환경을 고려한 관리 중심의 설계까지 포함하고 있다고 볼 수 있다.